Fakultät 1
Fachgebiet Praktische
Informatik/Softwaresystemtechnik
Prof. Dr. rer. nat. Leen Lambers

Brandenburgische
Technische Universität
Cottbus - Senftenberg

Modul 12202 - Softwarepraktikum
Wintersemester 2025/2026

Gruppe A

Produktdokumentation

vorgelegt von:

Autor	Matrikelnummer
Marwa Al Siyamji Al Mousli	5003322
Valentina Aseeva	5006672
Bruno Schreier	5007043
Eric Staudt	5012775
Hai Hoang Du	5013508

Cottbus, den 29. Januar 2026

Tutor: Leonie Ziechmann

Inhaltsverzeichnis

1 Anforderungsspezifikation und Beschreibung der Systemtests 5
1.1 Qualitätsszenarien . . . 5
1.2 Glossar . . . 7
1.2.1 Fachbegriffe . . . 7
1.2.2 Prozessbegriffe . . . 8
1.3 Domänenstruktur . . . 10
1.4 Geschäftsprozess . . . 11
1.5 Systemfunktionen . . . 12
1.5.1 Übersicht . . . 12
1.5.2 Beschreibung pro Use Case . . . 13
1.6 Beschreibung Systemtests . . . 17

2 Architekturbeschreibung und Beschreibung Integrationstests 18
2.1 Bausteinschicht . . . 18
2.1.1 Komponentendiagramm . . . 18
2.1.2 Klassendiagramm . . . 21
2.2 Laufzeitsicht . . . 22
2.2.1 Anmeldung . . . 22
2.2.2 Smartwatch Koppeln . . . 26
2.2.3 Schlafprotokoll ausfüllen . . . 27
2.2.4 Bericht ansehen . . . 28
2.2.5 Patientenliste ansehen und Patientendetails anzeigen . . . 29
2.3 Verteilungssicht . . . 30
2.3.1 Deploymentdiagramm . . . 30
2.4 Integrationstests . . . 30

3 Dokumentation der Testdurchführung 33
3.1 Android Unit-Tests . . . 33
3.1.1 AlleApiTests.kt . . . 33
3.1.2 AlleDataServiceTests.kt . . . 34
3.1.3 AlleHelperTests.kt . . . 37
3.2 Android Integrationstests . . . 38
3.2.1 AuthApiIntegrationTest.kt . . . 39
3.2.2 ProtocolApiIntegrationTest.kt . . . 40
3.2.3 ReportApiIntegrationTest.kt . . . 40
3.3 Backend-Modultests des Schlaftagebuch-Systems (Spring Boot) . . . 41
3.3.1 Getestete Module und Use Cases . . . 41
3.3.2 Teststrategie . . . 41
3.3.3 Testszenarien . . . 41
3.3.4 Technische Infrastruktur . . . 42
3.3.5 Ausführung . . . 42
3.3.6 Testergebnisse . . . 43

3.4 Test mit Postman . . . 45
3.4.1 Als Arzt einloggen . . . 45
3.4.2 Zugangscode generieren . . . 46
3.4.3 Erster Login des Patienten . . . 46
3.4.4 Test-Login Patient (über Seed, DEV) . . . 46
3.4.5 Protokoll absenden . . . 47
3.4.6 Bericht erstellen (PDF) – Test . . . 47
3.5 Query-Parameter . . . 48

4 Installations- und Bedienungsanleitung 50
4.1 Zielgruppe und Zweck . . . 50
4.2 Rollen und Berechtigungen (Kurzüberblick) . . . 50
4.3 Voraussetzungen (Software) . . . 50
4.4 Backend: Infrastruktur und Server starten . . . 51
4.5 Web-Anwendung . . . 53
4.6 Android: App starten und Grundfunktion prüfen . . . 54
4.7 Smartwatch / Wear OS (optional) . . . 55

5 Quellenangabe (Informationsbeschaffung / Dokumentation) 56
5.1 Verwendete Tools und Programme . . . 57

Abbildungsverzeichnis

1 Domänenstruktur: Schlaftagebuch . . . 10
2 Geschäftsprozessdiagramm: Schlaftagebuch . . . 11
3 Use-Case-Diagramm: Schlaftagebuch . . . 12
4 Systemtests . . . 17
5 Komponentendiagramm . . . 18
6 Komponent Systembackend . . . 19
7 Klassendiagramm . . . 21
8 Anmeldung für Arzt . . . 22
9 Anmeldung für Patient . . . 23
10 Konto erstellen für Patient . . . 24
11 Zugangscode in Arzt Web-GUI erstellen . . . 25
12 Smartwatch koppeln: Datenmessung starten . . . 26
13 Smartwatch koppeln: Datenmessung beenden . . . 26
14 Schlafprotokoll ausfüllen (und auf dem Server speichern) . . . 27
15 Bericht ansehen für Patientin . . . 28
16 Bericht ansehen für Arzt . . . 28
17 Patientenliste ansehen für Arzt . . . 29
18 Patientendetails für Arzt . . . 29
19 Deploymentdiagramm . . . 30
20 Erfolgreicher Start der Datenbank über Docker Compose . . . 51

21 Erfolgreicher Start des Backends mit laufendem `bootRun`-Prozess 52

1 Anforderungsspezifikation und Beschreibung der Systemtests

Die Software dient der digitalen Erfassung des Schlafverhaltens von Jugendlichen ab 14 Jahren, um Ärzt:innen vor einer Untersuchung im Kinderschlaflabor verlässliche Daten bereitzustellen. Sie ersetzt unzuverlässige Papierfragebögen durch eine mobile App, in der Jugendliche täglich ein Morgenprotokoll ausfüllen. Optional können Smartwatch-Daten integriert werden. Die Ergebnisse werden automatisch ausgewertet und als PDF- oder Excel-Bericht an das medizinische Personal übermittelt.
Randbedingungen und Qualitätsziele:

- **Datenschutz**: Nutzung nur mit Einverständnis; Zugriff ausschließlich für Jugendliche und Ärzt:innen.
- **Mehrsprachigkeit**: Deutsch, Englisch, weitere Sprachen wählbar.
- **Benutzerfreundlichkeit**: Einfache, intuitive Bedienung (max. 5 Minuten täglich).
- **Zuverlässigkeit und Sicherheit**: Verschlüsselte Datenspeicherung, fehlerfreie Übertragung.

1.1 Qualitätsszenarien

Datenschutz / Privacy

Quelle: Systembetrieb

Stimulus: Speicherung und Verarbeitung personenbezogener Daten (z. B. Name, Geburtsdatum).

Artefakt: Backend und Datenbank

Umgebung: Produktivbetrieb.

Reaktion: Es werden ausschließlich notwendige personenbezogene Daten gespeichert. Der Zugriff auf Patientendaten ist strikt rollenbasiert und erfolgt ausschließlich über gültige Authentifizierungs-Tokens (JWT).

Messgröße: Kein Zugriff auf fremde Patientendaten; Einhaltung der Datenminimierung.

Benutzerfreundlichkeit

Quelle: Patient:in (Jugendliche/r ab 14 Jahren)

Stimulus: Öffnet die App am Morgen, um das tägliche Schlafprotokoll auszufüllen.

Artefakt: Android-Anwendung (Patient:in GUI)

Umgebung: Normalbetrieb, mobile Nutzung.

Reaktion: Die App führt Schritt für Schritt durch das Protokoll, validiert Eingaben und bestätigt die erfolgreiche Speicherung.

Messgröße: Das Ausfüllen des Protokolls ist in ungefähr 5 Minuten möglich; Eingabefehler werden verständlich angezeigt.

Zuverlässigkeit / Offline-Fähigkeit

Quelle: Netzwerk / Infrastruktur

Stimulus: Patient:in sendet ein Schlafprotokoll, während Backend oder Netzwerk nicht verfügbar ist.

Artefakt: Android-App und Backend-API

Umgebung: Offline-Betrieb oder instabile Verbindung.

Reaktion: Das Protokoll wird lokal gespeichert und kann nach Wiederherstellung der Verbindung erneut gesendet werden.

Messgröße: Kein Datenverlust; lokale Speicherung bleibt nach App-Neustart erhalten.

Sicherheit / Authentifizierung

Quelle: Unautorisierte Nutzer:in oder abgelaufene Sitzung

Stimulus: Zugriff auf geschützte API-Endpunkte ohne gültigen JWT-Token.

Artefakt: Systembackend (Auth-, Report- und Management-Module)

Umgebung: Normalbetrieb.

Reaktion: Der Zugriff wird verweigert; das System liefert einen entsprechenden Fehlercode (401/403).

Messgröße: 100 % der nicht autorisierten Zugriffe werden blockiert; keine sensiblen Daten werden ausgeliefert.

Performance / Berichtserstellung

Quelle: Ärzt:in oder Patient:in

Stimulus: Anforderung eines PDF-Berichts über einen definierten Zeitraum (z. B. 14 Tage).

Artefakt: Backend (Report-Service)

Umgebung: Normalbetrieb, lokale Demonstrationsumgebung.

Reaktion: Der Bericht wird generiert und als PDF ausgeliefert oder es erfolgt eine Fehlermeldung ohne Systemabsturz.

Messgröße: Die PDF-Generierung erfolgt innerhalb von maximal 5 Sekunden unter Normalbedingungen; es treten keine ungefangenen Exceptions oder Systemabstürze auf.

1.2 Glossar

1.2.1 Fachbegriffe

Patient:in — Nutzer:in der Anwendung, die/der täglich ein Schlafprotokoll ausfüllt. Verantwortlich für das optionale Koppeln einer Smartwatch.

Ärzt:in — Medizinisches Fachpersonal, das die exportierten Berichte zur Auswertung erhält.

Nutzer:in — Eine Person, die das System verwendet. Der Begriff umfasst sowohl Patient:innen als auch Ärzt:innen.

Patient:innen nutzen das System zur täglichen Eingabe und Verwaltung ihrer Schlafdaten.

Ärzt:innen verwenden das System, um Berichte einzusehen und die Schlafdiagnostik zu unterstützen.

Nutzerkonto — Persönliches App-Konto einer Patient:in oder Ärzt:in, das mit einem einmaligen Zugangscode aktiviert wird.

Personenbezogene Daten (PbD) — Alle Informationen, die eine natürliche Person eindeutig identifizieren oder identifizierbar machen.

Im Kontext unseres Schlaf-Tagebuchs gehören dazu ausschließlich die Stammdaten der Nutzer:innen:

- Vorname
- Nachname
- Geburtsdatum

Diese Daten dienen ausschließlich der Zuordnung eines Nutzerkontos zu einer konkreten Person und sind gemäß DSGVO besonders zu schützen.

Erziehungsberechtigte — Gesetzliche Vertretung einer minderjährigen Patient:in; ab 18 übernimmt die/der Patient:in diese Rolle selbst.

Schlaftagebuch — Digitale Anwendung zur täglichen Dokumentation des Schlafverhaltens. Speichert, verwaltet und synchronisiert Schlafdaten und Protokolle und steuert den Anmeldevorgang. Leitet Daten an Berichte weiter und ermöglicht den Export.

Einwilligung zur Datenverarbeitung — Zustimmung zur Erhebung, Speicherung und Verarbeitung personenbezogener Gesundheitsdaten gemäß DSGVO.

Einmaliger Zugangscode/Zugangscode/Einmalcode — Von der Klinik ausgegebener Code zur erstmaligen Anmeldung und Aktivierung des Nutzerkontos.

Anmeldung — Zugang zur App und Aktivierung des Nutzerkontos; umfasst auch die Verwaltung und Änderung der Zugangscodes.

Benachrichtigung — Tägliche Erinnerung der App an das Ausfüllen des Schlafprotokolls.

Benachrichtigungszeit — Von der Patient:in gewählte Uhrzeit für die tägliche Erinnerung.

Schlafprotokoll — Fragebogen zur Selbsteinschätzung der gesundheitlichen und emotionalen Verfassung, zur Schlafsituation sowie zur Erfassung der Schlafdauer.

Schlafdauer — Automatisch berechnete Zeitspanne zwischen Einschlafzeit und Aufwachzeit.

Smartwatch — Tragbares Gerät, das Schlafdaten automatisch erfasst und mit der App synchronisiert; optional.

Schlafdaten — Von der Smartwatch gesammelte Daten der Nacht; optional und werden vom Schlaftagebuch verwaltet.

Synchronisation — Abgleich der in der App eingegebenen Daten mit automatisch erfassten Smartwatch-Daten.

Visualisierung — Grafische Darstellung der erfassten Schlafdaten.

Bericht / Dokument — Automatisch generiertes Dokument, das die gespeicherten Schlafdaten zusammenfasst.

Generierung / Erstellung — Automatischer Prozess der Berichtserzeugung.

Export — Bereitstellung des Berichts zum Herunterladen oder sicheren Übermitteln an die Klinik.

1.2.2 Prozessbegriffe

Einmaligen Zugangscode erhalten — Empfang des von der Klinik bereitgestellten Codes zur Aktivierung des Nutzerkontos.

Anmelden — Erstellung eines Nutzerkontos mit klinischen Zugangsdaten; umfasst auch deren Änderung.

Einwilligung erteilen — Zustimmung zur Datenverarbeitung innerhalb der App.

Einstellungen verwalten — Übergeordneter Begriff für Spracheinstellung und Benachrichtigungszeit.

Sprache auswählen — Festlegung der App-Sprache (Deutsch, Englisch, Russisch).

Benachrichtigungszeit auswählen — Festlegung der Uhrzeit für die tägliche Erinnerung.

Benachrichtigung erhalten — Empfang der Erinnerung zum Ausfüllen des Schlafprotokolls.

Smartwatch koppeln — Verbinden der App mit einer kompatiblen Smartwatch zur Synchronisation.

Daten synchronisieren — Abgleich der manuell eingegebenen Informationen mit Smartwatch-Daten.

Visualisierung ansehen — Öffnen und Betrachten der grafischen Darstellung der Schlafdaten.

Schlafprotokoll ausfüllen — Beantworten der Fragen zum Schlafverhalten und Tagesbefinden.

Bericht erstellen / generieren — Automatische Erstellung eines Berichts aus allen vorhandenen Daten.

Daten exportieren — Übermittlung des Berichts an die/den Ärzt:in und Speichern im Protokoll.

1.3 Domänenstruktur

Die Domänenstruktur des digitalen Schlaftagebuchs beschreibt die wichtigsten Beteiligten (Patient:in, Ärzt:in), zentrale Artefakte (Schlafprotokoll, Bericht) sowie deren Beziehungen. Abbildung 1 zeigt diese Struktur als Klassendiagramm.

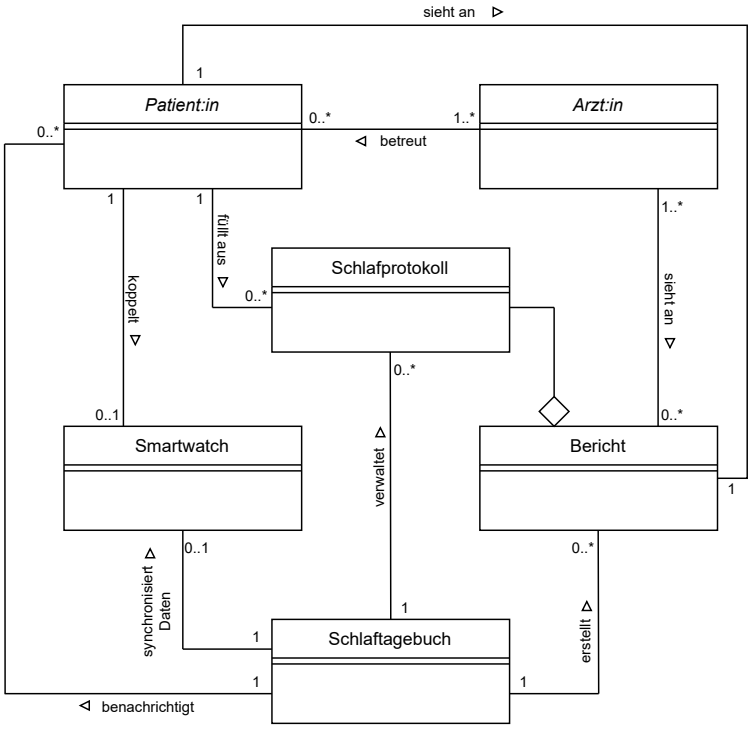


Abbildung 1: Domänenstruktur: Schlaftagebuch

Eine Patient:in füllt regelmäßig Schlafprotokolle aus; jedes Protokoll gehört genau einer Patient:in (0..* → 1), während eine Patient:in beliebig viele Protokolle besitzen kann (Im besten Fall aber nur 14). Diese Protokolle werden im Schlaftagebuch gesammelt.

Optional kann eine Smartwatch mit einer Patient:in gekoppelt werden (Patient:in 1 → 0..1 Smartwatch). Sie stellt automatisch erfasste Schlafdaten bereit, die mit dem Schlaftagebuch synchronisiert werden können.

Aus einem oder mehreren Schlafprotokollen wird ein Bericht erstellt. Die Aggregationsbeziehung zeigt, dass ein Bericht Protokolle zusammenfasst, die jedoch unabhängig weiterexistieren können. Eine Ärzt:in erhält diese Berichte zur Auswertung und betreut die jeweilige Patient:in (Ärzt:in 1 → 0..* Patient:innen).

1.4 Geschäftsprozess

Der Geschäftsprozess beschreibt den Ablauf der Schlafdiagnostik für Jugendliche im Schlaflabor. Ziel ist es, Schlafstörungen zu erkennen und eine Diagnose zu stellen. Die Software unterstützt diesen Prozess, indem sie digitale Schlafprotokolle sammelt, vervollständigt, validiert und in einem automatisch generierten Bericht zusammenfasst. Ärzt:innen nutzen diesen Bericht zur Auswertung und zur Erstellung einer Diagnose.

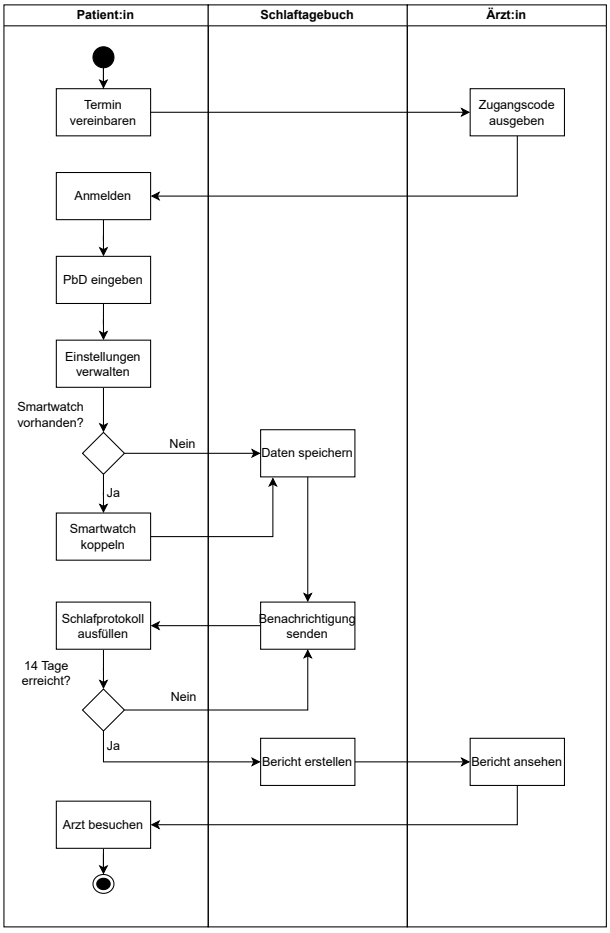


Abbildung 2: Geschäftsprozessdiagramm: Schlaftagebuch

1.5 Systemfunktionen

1.5.1 Übersicht

Die dargestellte Use-Case-Diagramm zeigt die zentralen Systemfunktionen des Schlaftagebuchs aus der Perspektive der beiden Hauptakteure: **Patient:in** und **Arzt:in**. Es veranschaulicht, welche Aktionen die jeweiligen Nutzergruppen mit dem System durchführen können und wie sie mit den Kernfunktionalitäten interagieren.

Patient:innen sind die primären Anwender:innen des Systems. Sie können sich im Schlaftagebuch anmelden, ihre Einstellungen verwalten und eine Smartwatch mit dem System koppeln, um Schlafdaten automatisiert zu erfassen. Darüber hinaus haben Patient:innen die Möglichkeit, täglich ein Schlafprotokoll auszufüllen, in dem subjektive und objektive Angaben zum Schlafverhalten erfasst werden.

Ärzt:innen nutzen das System zur Verwaltung ihrer Patient:innen. Sie können Zugangscodes für Patient:innen erstellen, haben Zugriff auf Informationen von den Patient:innen und ihnen steht eine Berichtsübersicht zur Verfügung, die alle erstellten PDF-Berichte der zugeordneten Patient:innen umfasst.

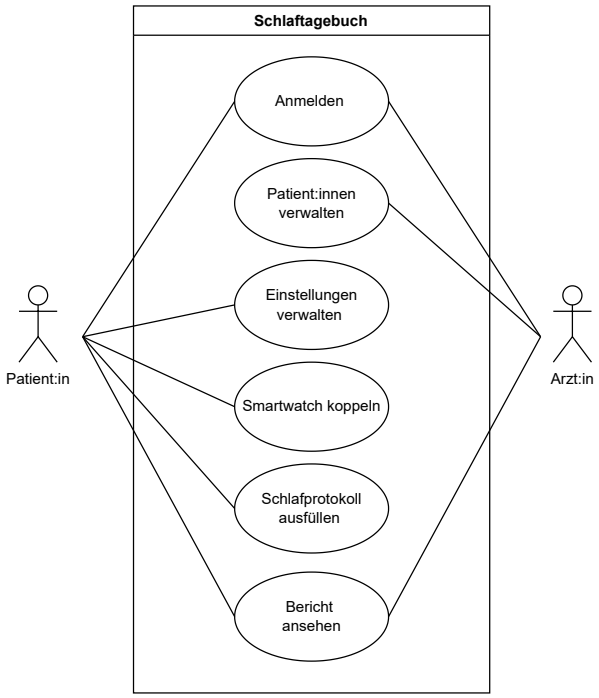


Abbildung 3: Use-Case-Diagramm: Schlaftagebuch

1.5.2 Beschreibung pro Use Case

Use Case: Anmelden

Titel: Anmelden

Kurzbeschreibung: Aktivierung und Anmeldung des Nutzerkontos.

Aktoren: Nutzer:in (Patient:in oder Ärzt:in)

Vorbedingungen: Patient:in: Ein einmaliger Zugangscode liegt vor.

Ärzt:in: Anmeldedaten von IT-Personal (Username, Passwort)

Ablauf:

Patient:in:

1. Gibt den Zugangscode und PbD ein, erstellt ein eigenes Passwort.

2. System aktiviert das Nutzerkonto.

3. Angemeldet.

Ärzt:in:

1. Gibt Username und Passwort ein.

2. Angemeldet.

Auswirkungen: Nutzerkonto wird vollständig nutzbar.

Anmerkungen: Für Patientin:nen wird ein Username erstellt, die sie (mit eigenem Passwort) für künftige Login ohne Zugangscode nutzen können.

Use Case: Patient:innen verwalten

Titel: Patient:innen verwalten

Kurzbeschreibung: Ärzt:innen verwalten Patient:innen, erstellen Zugangscodes für die Registrierung und können Patientendaten ansehen.

Aktoren: Ärzt:in

Vorbedingungen: Ärzt:in ist im System angemeldet.

Ablauf:

1. Ärzt:in öffnet die Übersicht (Dashboard).

2. System zeigt eine Übersicht, z. B. Anzahl der Patient:innen und Anzahl der verfügbaren Berichte.

3. Ärzt:in öffnet die Seite zur Generierung von Zugangscodes.

4. Ärzt:in erstellt einen neuen Zugangscode für eine:n Patient:in.

5. System speichert den Zugangscode und zeigt ihn in der Liste aller generierten Codes an.

6. Ärzt:in öffnet die Patientenübersicht.

7. Ärzt:in wählt eine:n Patient:in aus.

8. System zeigt grundlegende Patientendaten.

Auswirkungen: Zugangscodes werden gespeichert und stehen für die Registrierung zur Verfügung. Ärzt:innen können Patient:innen einsehen.

Use Case: Einstellungen verwalten

Titel: Einstellungen verwalten

Kurzbeschreibung: Änderung der Sprache und der täglichen Benachrichtigungszeit.

Aktoren: Patient:in

Vorbedingungen: Nutzerkonto ist angemeldet.

Ablauf:

1. Öffnen des Einstellungsbereichs.

2. Auswahl der gewünschten Sprache.

3. Festlegung der täglichen Erinnerungszeit.

4. System speichert die neuen Einstellungen.

Auswirkungen: App-Sprache und Benachrichtigungen passen sich an.

Anmerkungen: Einstellungen können jederzeit geändert werden.

Use Case: Smartwatch koppeln

Titel: Smartwatch koppeln

Kurzbeschreibung: Verbindung der App mit einer kompatiblen Smartwatch zur Sendung von Start und Stopp signalen der Sammlung von Schlafdaten sowie Synchronisation dieser gesammelten Daten.

Aktoren: Patient:in

Vorbedingungen: Eine Smartwatch ist vorhanden und eingeschaltet.

Ablauf:

1. Patient:in verbindet die Smartwatch mit dem Handy (außerhalb der App über Bluetooth, wenn nicht bereits verbunden).

2. Patient:in sendet ein Startsignal der Messung.

3. Die Smartwatch sammelt dauerhaft Herzfrequenz und Bewegungsdaten des Accelerometers.

3. Patient:in sendet ein Stoppsignal der Messung

4. Smartwatch sendet die gesammelten Daten an das Smartphone.

Auswirkungen: Daten von Smartwatch werden auf Wunsch erfasst und an die App übermittelt.

Use Case: Schlafprotokoll ausfüllen

Titel: Schlafprotokoll ausfüllen

Kurzbeschreibung: Tägliche Eingabe der subjektiven und objektiven Angaben zum Schlaf.

Aktoren: Patient:in

Vorbedingungen: Nutzerkonto ist angemeldet.

Ablauf:

1. Öffnen des Formulars für den aktuellen Tag.

2. Antworten auf die Fragen.

3. System speichert das Protokoll.

Auswirkungen: Das Tagesprotokoll ist vollständig gespeichert.

Anmerkungen: Pro Tag können mehrere Protokolle abgegeben werden. Gespeichert und für den Bericht verwendet wird jedoch nur das letzte Protokoll.

Use Case: Bericht ansehen

Titel: Bericht ansehen

Kurzbeschreibung: Der Bericht wird aus den Protokollen erstellt und zur Ansicht bereitgestellt.

Aktoren: Patient:in, Ärzt:in

Vorbedingungen: Nutzer ist autorisiert. Patient hat mindestens ein Protokoll ausgefüllt.

Ablauf:

Ärzt:in:

1. Ärzt:in öffnet die Berichtsübersicht.

2. Auswahl eines gewünschten Berichts.

3. System lädt und zeigt das Dokument.

Patient:in:

1. Der Knopf „Bericht herunterladen“ wird angeklickt.

2. System lädt und zeigt das Dokument.

Auswirkungen: Bericht kann ausgewertet werden.

1.6 Beschreibung Systemtests

- **Ein Systemtest pro Use Case** für den Standardablauf
- Zusätzliche Tests für **alternative Abläufe**

Use-Case	Titel	Input	Output	Fehler
Anmelden	Zugangscode eingeben	App installieren und öffnen, dann Zugangscode vom Arzt eingebn	Nutzer wird zur Eingabe des Passworts weitergeleitet	Server-Probleme, falscher Zugangscode, Zugriff wird verweigert
	Passwort eingeben	Passwort eingeben	Passwort ist gültig und Nutzer wird eingeloggt/ein Nutzerkonto erstellt	Passwort ist falsch, fälschlicherweise eingeloggt
Einstellung verwalten	Sprache auswählen	Nutzer wählt die Sprache aus, die er/sie bevorzugt	Sprache in der App ist nun zur ausgewählten Spache gewechselt	Sprachübersetzung funktioniert nicht richtig, Nutzer hat die falsche Sprache ausgewählt
Smartwatch koppeln	Smartwatch koppeln	Nutzer koppelt optional eine Smartwatch mit der App	Smartwatch ist nun gekoppelt mit App und nimmt Schlafdaten auf	Keine Smartwatch vorhanden, Verbindung schlecht
Schlafprotokoll ausfüllen	Schlafdaten speichern	Eingegebene Daten endgültig speichern	Daten werden final auf der Datenbank gespeichert	Datenbank ist nicht erreichbar, Serverfehler
	Zeitangaben tätigen	Uhrzeit mit Stunden und Minuten angeben	Zeitangabe wird in der App gespeichert	Es wird eine ungültige Zeitangabe getätigt
	Schlafprotokoll auslassen	Schlafprotokoll wird ausgelassen, weil der Nutzer es vergessen hat, er/sie keine zeit oder keine Lust hat	Daten für diesen Tag nicht vorhanden	Ungenügend Daten nach 14 Tagen für den Bericht
Bericht ansehen	Bericht wird erstellt	Bericht wird erstellt mit den Daten über die letzten 14 Tage	Bericht wird generiert und kann angesehen werden	Fehler bei Verarbeitung der Daten (ungültige Daten, nicht genügend Daten, ...)
	Bericht anzeigen	Nutzer ruft den Bericht in der App auf	Bericht wird dem Nutzer angezeigt (Schlafdaten und generierte Visualisierung)	Bericht wird nicht vollständig oder gar nicht generiert

Abbildung 4: Systemtests

2 Architekturbeschreibung und Beschreibung Integrationstests

2.1 Bausteinschicht

2.1.1 Komponentendiagramm

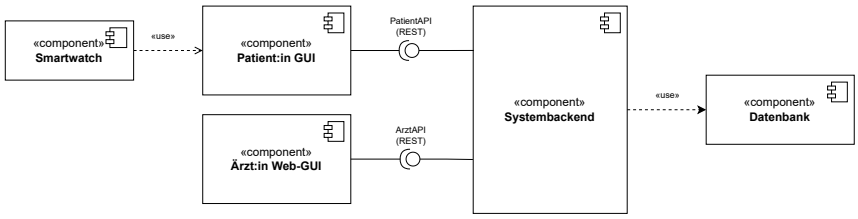


Abbildung 5: Komponentendiagramm

Die Systemübersicht zeigt die Hauptkomponenten der Anwendung und deren Interaktion.

Smartwatch
Die Verbindung zur Smartwatch dient ausschließlich der Datenerfassung; die Smartwatch selbst greift nicht direkt auf das Backend zu.

Patient:in GUI
Das Patient:in GUI stellt die mobile Benutzeroberfläche für Patient:innen dar. Es kommuniziert über die REST-Schnittstelle PatientAPI mit dem Systembackend, um schlafbezogene Daten zu übertragen und Funktionen des Systems zu nutzen.

Ärzt:in Web-GUI
Das Ärzt:in Web-GUI stellt die Weboberfläche für medizinisches Fachpersonal dar. Es kommuniziert ebenfalls über eine REST-Schnittstelle (ArztAPI) mit dem Systembackend und ermöglicht den Zugriff auf ärztliche Funktionen wie die Verwaltung von Patient:innen und die Einsicht in Berichte.

Systembackend
Das Systembackend stellt die notwendigen REST-Schnittstellen für das Patient:in GUI und das Ärzt:in Web-GUI bereit und ist zuständig für die Verarbeitung und persistente Speicherung solcher Daten, die sowohl für Patient:innen als auch für Ärzt:innen zugänglich sein müssen, z. B. Zugangscodes, übermittelte Schlafprotokolle und daraus generierte Berichte. Lokale Daten, wie benutzerspezifische Einstellungen, Spracheinstellungen oder direkt von der Smartwatch erfasste Rohdaten, werden ausschließlich im jeweiligen Frontend verarbeitet und gespeichert und sind nicht Bestandteil der serverseitigen Verarbeitung.

Datenbank
Die Datenbank dient der persistenten Speicherung der Anwendungsdaten. Der Zugriff auf die Datenbank erfolgt ausschließlich über das Systembackend.

Komponent Systembackend (Detaliiert)

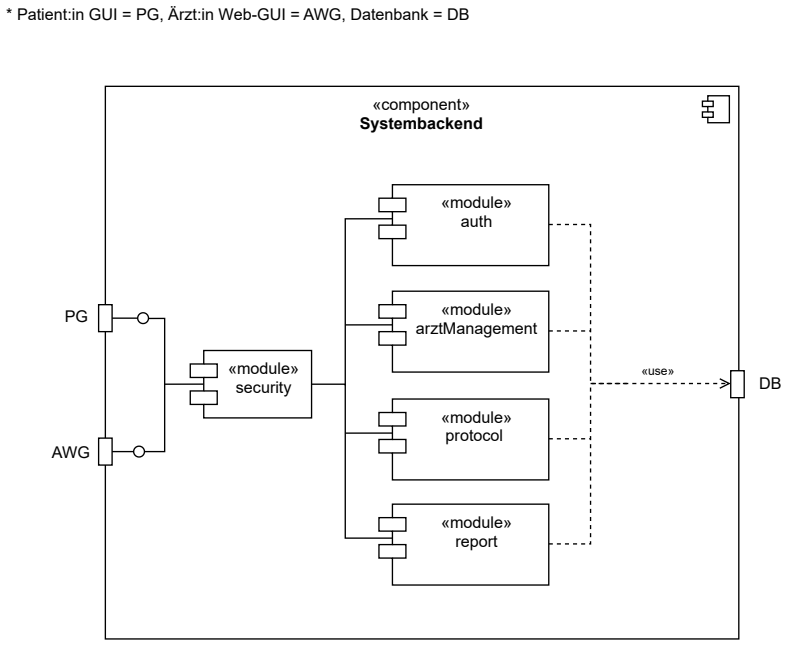


Abbildung 6: Komponent Systembackend

Die detaillierte Komponentendiagramm zeigt die interne Struktur des Systembackends. Der Zugriff auf die bereitgestellten REST-Schnittstellen erfolgt über die Ports PG (PatientAPI) und AWG (ArztAPI). Alle eingehenden Anfragen werden zunächst durch das Modul security verarbeitet. Dieses Modul ist für Authentifizierung, Token-Validierung und Rollenprüfung zuständig und wirkt als vorgelagerte Sicherheitskomponente vor der Fachlogik. Die fachliche Verarbeitung erfolgt in den folgenden Modulen:

• auth: Verwaltung von Benutzerkonten, Rollen und Authentifizierungsprozessen.

• arztManagement: Bereitstellung ärztlicher Funktionen zur Verwaltung von Patient:innen und zugehörigen Daten.

• protocol: Verarbeitung und Verwaltung der von Patient:innen erfassten Schlafprotokolle.

• report: Erzeugung und Bereitstellung von Schlafberichten auf Basis der gespeicherten Protokolldaten.

Der Zugriff auf die Datenbank erfolgt zentral über Repositories innerhalb des Backends.

Anmerkung bezüglich FHIR:

Im Projekt ist ein FHIR-Modul als Grundlage für eine spätere Anbindung an einen externen HL7-FHIR-Server implementiert.

Aktuell dient dieses Modul als technisches Template und kapselt die FHIR-spezifische Logik (z. B. die Abbildung interner Domänenobjekte auf FHIR-Ressourcen wie QuestionnaireResponse, Consent oder DiagnosticReport sowie die Verwendung von Referenzen wie Patient/<UUID> und Practitioner/<UUID> für Benutzerkonten als subjectRef).

Die FHIR-Daten werden derzeit ausschließlich in-memory verarbeitet, da kein externer FHIR-Server angebunden ist. Wesentliche Patientendaten (PatientEntity), Berichte (ReportEntity) sowie Einwilligungen (ConsentEntity) werden jedoch unabhängig davon noch dauerhaft in der PostgreSQL-Datenbank gespeichert.

Konzeptionell ist das FHIR-Modul als separater externer Dienst vorgesehen, der auf einem eigenen Server betrieben wird. In einer erweiterten Ausbaustufe würde dieser Dienst als eigenständige Komponente in der Systemarchitektur modelliert und über eine REST-basierte Schnittstelle angebunden werden. Da diese Infrastruktur im aktuellen Projektumfang nicht realisiert wurde, ist das FHIR-Modul derzeit als interner Bestandteil des Systembackends implementiert.

2.1.2 Klassendiagramm

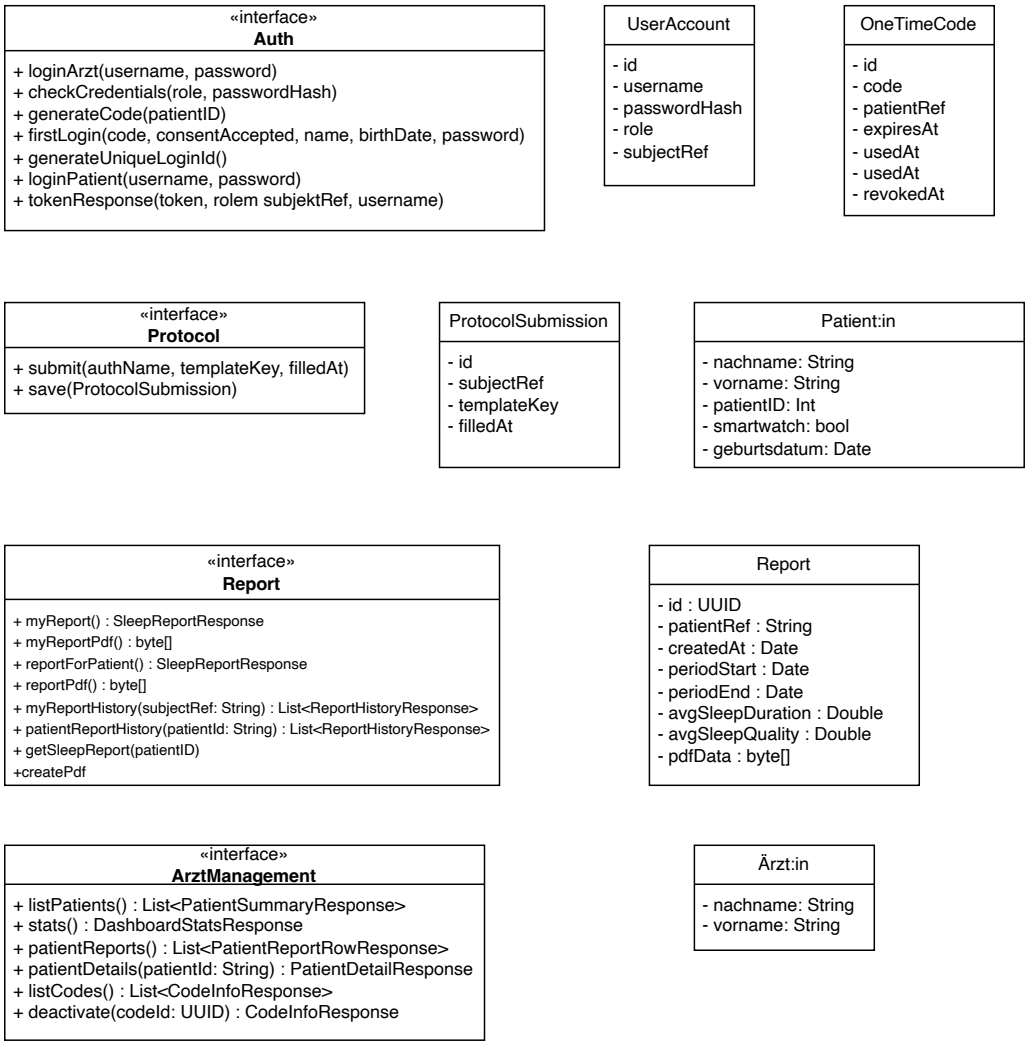


Abbildung 7: Klassendiagramm

2.2 Laufzeitsicht

Allgemeiner Hinweis zu den Sequenzdiagrammen In den folgenden Sequenzdiagrammen werden ausschließlich sogenannte *Happy-Path*-Szenarien dargestellt, um eine übersichtliche und verständliche Darstellung der Abläufe zu gewährleisten. Mögliche Fehlerfälle und Ausnahmebedingungen werden nicht grafisch modelliert, sondern ergänzend in der textuellen Beschreibung der jeweiligen Anwendungsfälle erläutert.

2.2.1 Anmeldung

Arzt-Login (Benutzername und Passwort)

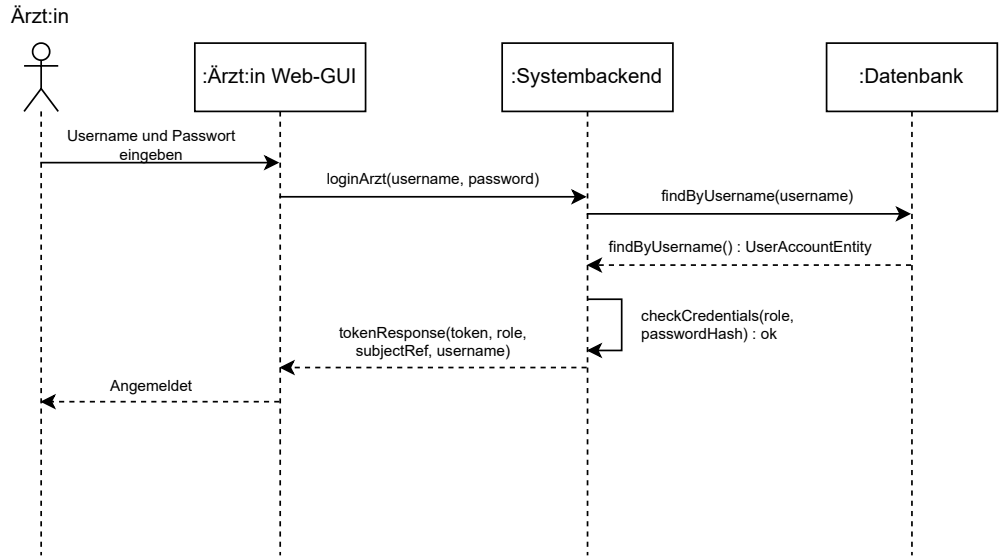


Abbildung 8: Anmeldung für Arzt

Anmerkungen:
Bei Auftreten eines der folgenden Fehlerfälle wird der Anmeldevorgang abgebrochen und es erfolgt keine Authentifizierung: **Der Benutzer existiert nicht** (falscher Benutzername), **das eingegebene Passwort ist falsch, ein technischer Fehler tritt auf**, z. B. wenn das Backend oder die Datenbank nicht erreichbar sind.

Patienten-Login (Benutzername und Passwort)

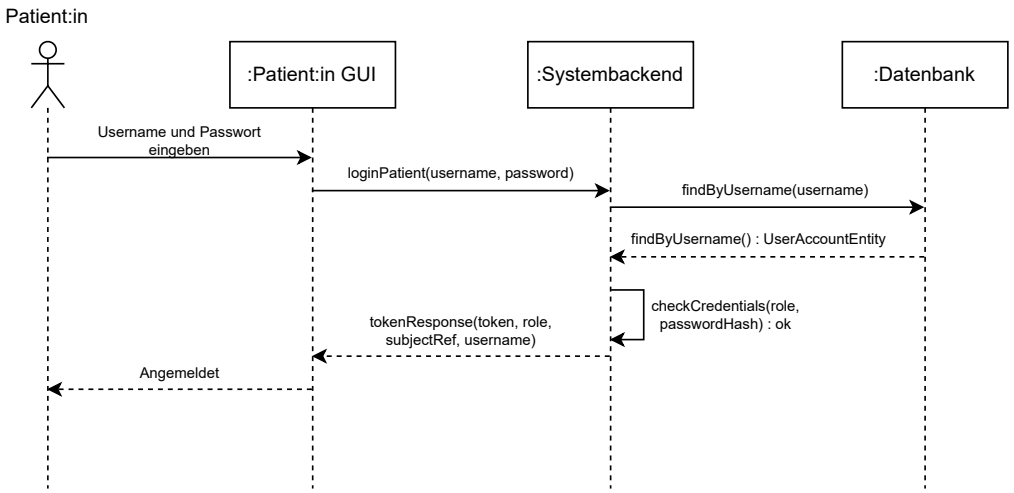


Abbildung 9: Anmeldung für Patient

Anmerkungen:
Bei Auftreten eines der folgenden Fehlerfälle wird der Anmeldevorgang abgebrochen und es erfolgt keine Authentifizierung: **Der Benutzer existiert nicht** (falscher Benutzername), **das eingegebene Passwort ist falsch, ein technischer Fehler tritt auf**, z. B. wenn das Backend oder die Datenbank nicht erreichbar sind.

Patienten-First Login (Zugangscode)

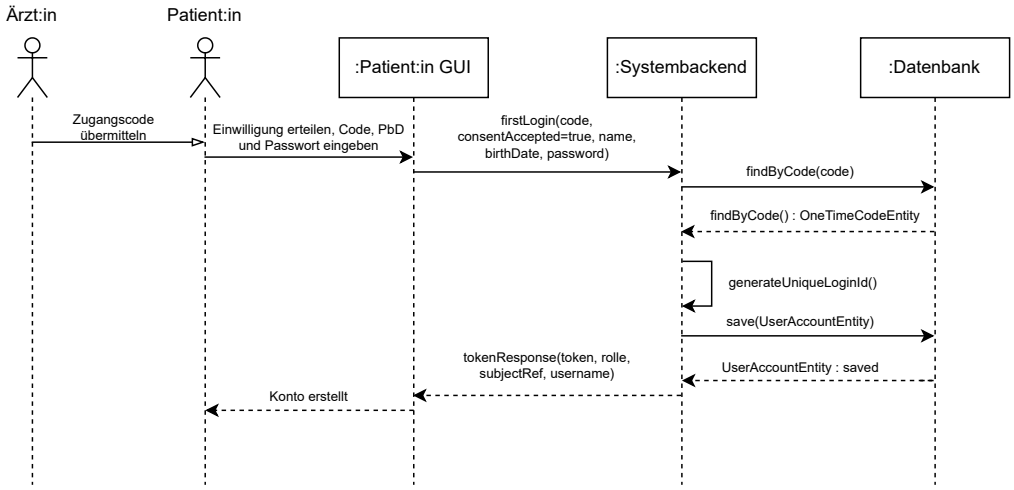


Abbildung 10: Konto erstellen für Patient

Anmerkungen:

Die Registrierung wird abgebrochen, wenn einer der folgenden Fehler auftritt:

Der eingegebene **Zugangscode ist ungültig** (z. B. falsch eingegeben oder bereits verwendet).

Die erforderliche **Einwilligung wurde nicht erteilt.** In diesem Fall ist der Button „Weiter“ deaktiviert und die Patientin bzw. der Patient erhält keinen Zugriff auf die App.

Die definierten **Eingaberegeln werden nicht erfüllt** (z. B. falsches Datumsformat `TT.MM.JJJJ` oder **nicht erfüllte Passwortregeln:** mindestens acht Zeichen, davon mindestens ein Buchstabe und eine Ziffer). In diesem Fall wird ein entsprechender Hinweis angezeigt und die Eingabe muss korrigiert werden. Ein **technischer Fehler tritt auf**, z. B. wenn das Backend oder die Datenbank nicht erreichbar sind.

Zusätzlich: **Zugangscode erstellen**

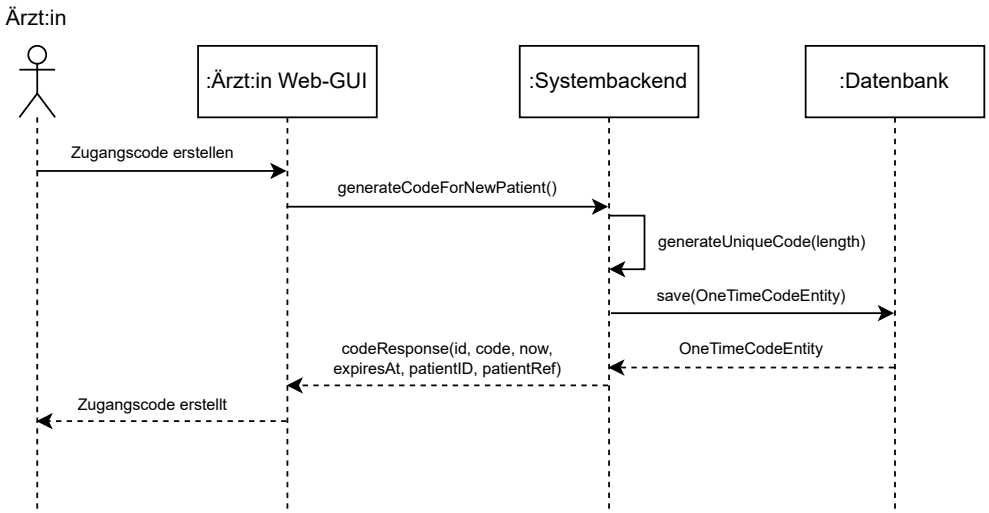


Abbildung 11: Zugangscode in Arzt Web-GUI erstellen

Anmerkungen:
Die Erstellung eines Zugangscodes erfolgt über die Web-GUI für Ärztinnen und Ärzte. Ohne einen gültigen Zugangscode ist eine Registrierung von Patientinnen und Patienten nicht möglich.

Der Vorgang kann nicht durchgeführt werden, wenn das **Backend oder die Datenbank nicht erreichbar sind**.

Das zugehörige Sequenzdiagramm beschreibt exemplarisch die Generierung eines Zugangscodes nur für neue Patientinnen und Patienten.

2.2.2 Smartwatch Koppeln

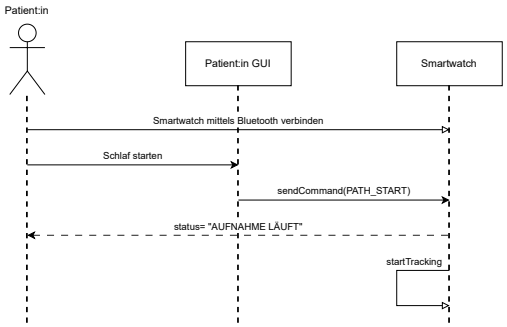


Abbildung 12: Smartwatch koppeln: Datenmessung starten

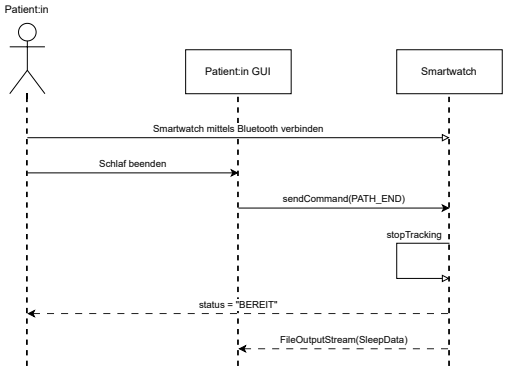


Abbildung 13: Smartwatch koppeln: Datenmessung beenden

Anmerkungen:
Folgende Fehler könnten in der Kommunikation auftreten und wurden berücksichtigt:

- **Verbindung:** Ohne bestehende Verbindung zwischen Smartwatch und Handy ist keine Steuerung über die App möglich. Bei einem Verbindungsabbruch wird die Kommunikation erst nach Wiederherstellung fortgesetzt.
- **Start und Beenden:** Befehle sind unabhängig voneinander. Ein Beenden vor dem Start ist wirkungslos; es werden keine Daten übertragen und keine Datei in der App erstellt.
- **Signalfolge:** Aufeinanderfolgende identische Signale (Start und Beenden) werden ignoriert. Es zählt jeweils nur das erste Signal.
- **Autonomie:** Die Smartwatch misst bei Verbindungsverlust weiter, sofern zuvor ein Start erfolgte. Nach erneuter Verbindung wirkt das Beenden Signal wie vorgesehen. Signale während Verbindungsverlust werden ignoriert.

2.2.3 Schlafprotokoll ausfüllen

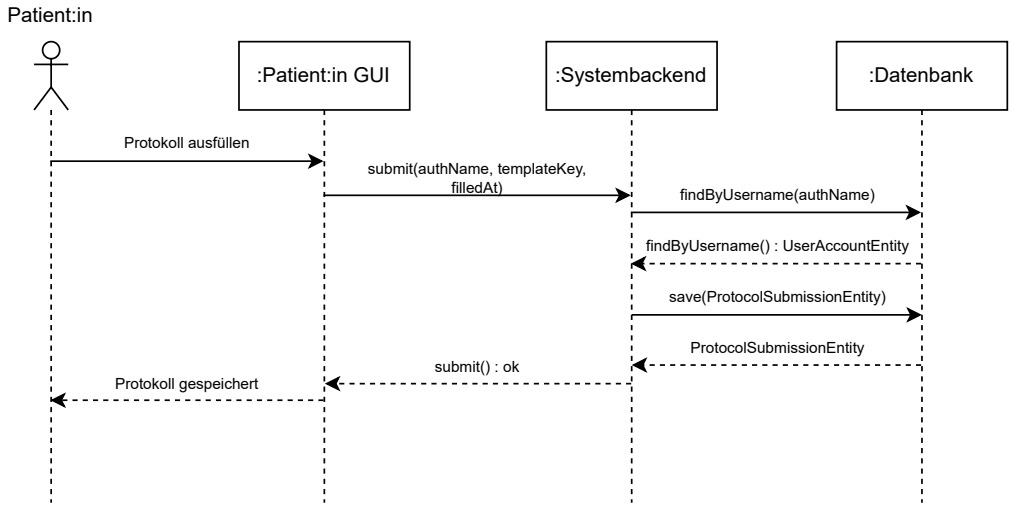


Abbildung 14: Schlafprotokoll ausfüllen (und auf dem Server speichern)

Anmerkungen:
Tritt während des Ausfüllens oder Übermittelns eines Protokolls ein Fehler auf, verhält sich das System wie folgt:

- Ist das **Backend oder die Datenbank nicht erreichbar**, wird das ausgefüllte Protokoll nicht an den Server übertragen, sondern ausschließlich lokal gespeichert.
- Ist der Fehler auf einen **ungültigen oder abgelaufenen Authentifizierungs-Token** zurückzuführen, wird der Benutzer zur erneuten Anmeldung weitergeleitet.

Bereits eingegebene Antworten werden in beiden Fällen lokal gespeichert und können nach erfolgreicher erneuter Authentifizierung erneut an den Server übermittelt werden.

2.2.4 Bericht ansehen

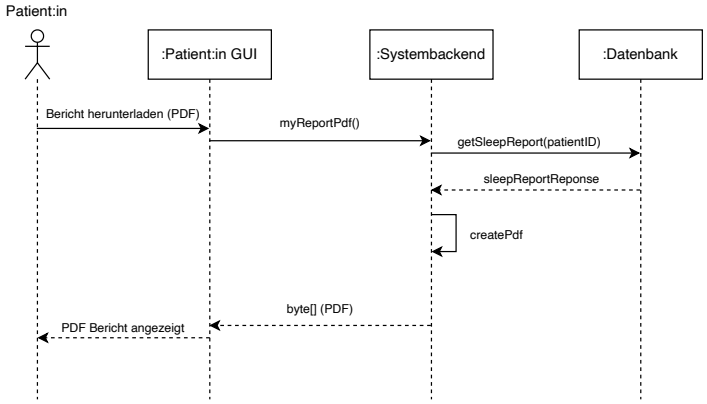


Abbildung 15: Bericht ansehen für Patientin

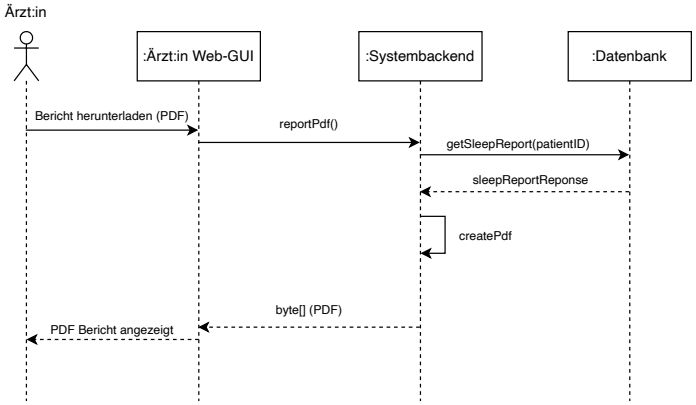


Abbildung 16: Bericht ansehen für Arzt

Anmerkungen:
Bei Auftreten eines der folgenden Fehlerfälle wird der Vorgang zum Anzeigen des Berichts abgebrochen und es erfolgt keine Anzeige des PDF-Berichts: Der Benutzer ist nicht authentifiziert oder die Sitzung ist abgelaufen, ein technischer Fehler tritt auf, z. B. wenn das Backend oder die Datenbank nicht erreichbar sind, oder es liegen keine Berichte für den angefragten Zeitraum vor.

2.2.5 Patientenliste ansehen und Patientendetails anzeigen

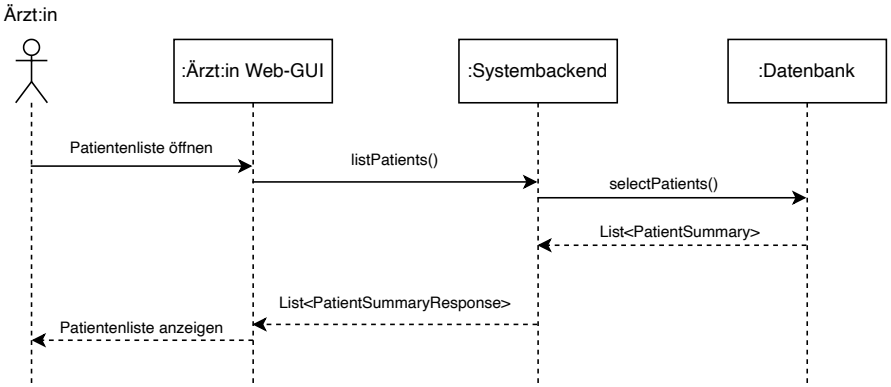


Abbildung 17: Patientenliste ansehen für Arzt

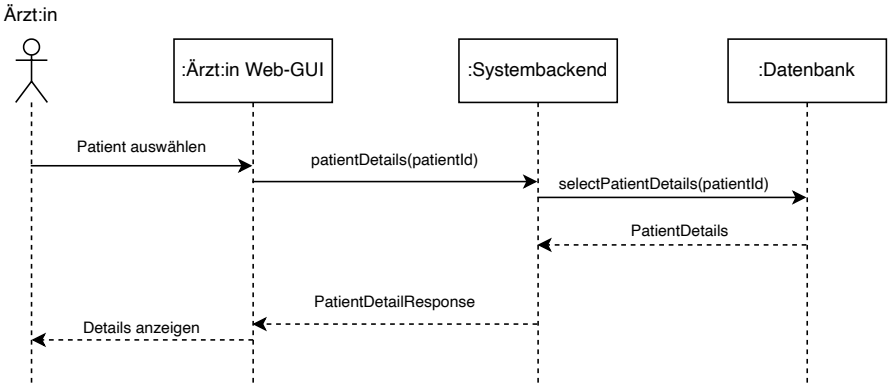


Abbildung 18: Patientendetails für Arzt

Anmerkungen:
Bei Auftreten eines der folgenden Fehlerfälle wird der Vorgang zum Anzeigen der Patientenliste bzw. der Patientendetails abgebrochen und es erfolgt keine Anzeige der entsprechenden Informationen: Die Ärztin oder der Arzt ist nicht authentifiziert oder nicht autorisiert, ein technischer Fehler tritt auf, z. B. wenn das Backend oder die Datenbank nicht erreichbar sind.

Zusätzlich kann der Vorgang zum Anzeigen der Patientendetails fehlschlagen, wenn der angefragte Patient nicht existiert oder der Ärztin bzw. dem Arzt nicht zugeordnet ist.

2.3 Verteilungssicht

2.3.1 Deploymentdiagramm

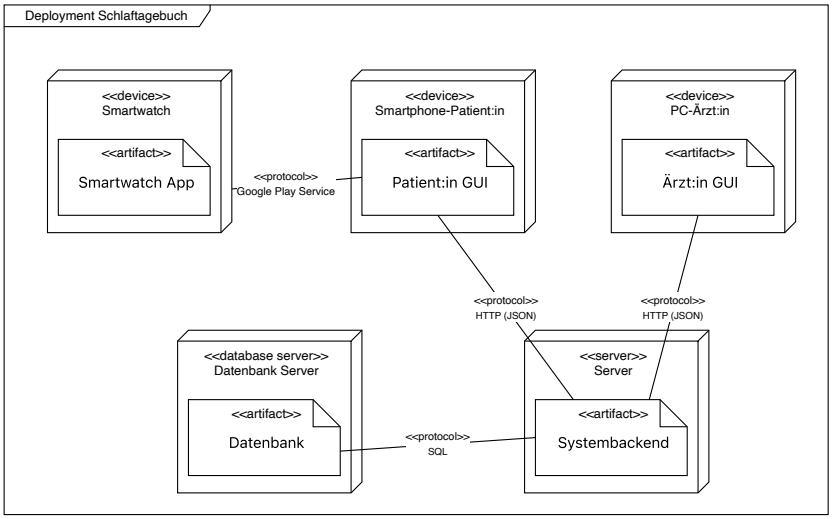


Abbildung 19: Deploymentdiagramm

2.4 Integrationstests

1. Schlafprotokoll ausfüllen

Test 1.1: Erfolgreich neues Schlafprotokoll speichern

Ziel	Prüfen, ob ein neuer Protokolleintrag korrekt von GUI
Vorbedingungen	Patient ist angemeldet
Eingaben (GUI)	Einschlafzeit: 22:30, Aufwachzeit: 07:10, Befunden: 4 und Datum wird automatisch eingetragen
Testschritte	1. Patient öffnet Protokollformular, GUI ruft *ausfüllenFragebogen()* auf 2. GUI sendet Daten an Backend via *neuenEintrag()* 3. Backend validiert Daten 4. Backend ruft DB-Methode *insert(Schlafprotokoll)* auf
Erwartete Backend-DB Interaktionen	*insert()* wird einmal mit vollständigem Datensatz ausgeführt
Erwartete Ausgabe (GUI)	„Eintrag erfolgreich erstellt"
Fehler	Keine

Test 1.2: Ungültige Schlafzeiten

Ziel	Überprüfung von eingetragenen Daten
Eingabe	Einschlafzeit: 23:00, Aufwachzeit: 22:00 (am selben Tag)
Erwartetes Ergebnis	Backend erkennt den Fehler, sendet eine Fehlermeldung an GUI wie z.B. „ungültige Zeitenünd *insert()* wird nicht ausgeführt.

2. Einwilligung erteilen

Test 2.1: Einwilligung vorhanden

Ziel	Prüfen, dass Einwilligung korrekt gespeichert wird
Vorbedingung	Patient gibt Zugangscode eine
Schritte	1. GUI → Backend: *informationErfragen()* 2. Backend → DB: *getConsentFormular(patientID)* 3. Patient klickt „Einverstanden"→ GUI → Backend: *einwilligungSpeichern()* 4. Backend → DB: *save(patientID)*
Erwartet	DB speichert *einverständnis=true*, GUI zeigt „Zugang freigeschaltet"

Test 2.2: Einwilligung verweigert

Ziel	Verhindern, dass Minderjährige ohne Zustimmung starten
Schritte	Gleich wie oben, aber Patient „Ablehnen"
Erwartet	DB speichert *false*, GUI zeigt „Zugang nicht möglich"

3. Bericht ansehen

Test 3.1: PDF wird Korrekt generiert

Ziel	Prüfen, ob Backend die Richtigen Daten holt und PDF generiert.
Schritte	1. Arzt wählt Patient 2. GUI → Backend: *berichtErstellen(patientID)* 3. Backend → DB: *getReport(patientID)* 4. Backend generiert PDF via *generatePDF()* 5. PDF an GUI senden
Erwartet	PDF enthält aller Einträge der Schlafprotokoll einer Patient

Test 3.2: Keine Einträge vorhanden

Kategorie	Beschreibung
Eingabe	PatientID existiert, aber keine Protokolle
Erwartet	Backend liefert eine PDF ohne Datensätze

4. Anmelden

Test 4.1: Login möglich

Ziel	Prüfen, dass Registrierung funktioniert
Schritte	1. GUI → Backend: *validateCode(code)* 2. Backend → DB: *checkCode()* 3. GUI → Backend: *setPassword()* 4. Backend → DB: *changePassword()*
Erwartet	Passwort gehasht gespeichert, Login möglich

Test 4.2: Kein Login möglich

Kategorie	Beschreibung
Eingabe	Ungültiger Code als Passwort eingeben
Erwartet	Fehlermeldung: „Ungültiger Code"kein Passwort darf gespeichert werden

5. Einstellung verwalten

Test 5.1: Sprache erfolgreich ändern

Schritte	GUI → Backend: *einstellungAnpassen(ËN")* Backend → DB: *updateUserSettings()* Backend lädt *loadLanguageResources()*
Erwartet	GUI wird vollständig auf Englisch aktualisiertwerden

Test 5.2: SprachPaket fehlt

Schritte	Gleich wie oben
Erwartet	Backend-Fehler → GUI: „Sprache nicht verfügbar"

Test 5.3: Erinnerung erfolgreich ändern

Schritte	GUI → Backend: *einstellungAnpassen(benachrichtigungsZeit)* Backend → DB: *updateUserSettings()* Backend setzt Timer
Erwartet	Neue Zeit erscheint im GUI

Test 5.4: Ungültige Zeit auswählen

Eingabe	Uhrzeit: 99:99
Erwartet	Fehlermeldung: „Ungültiger Code"kein Passwort darf gespeichert werden

3 Dokumentation der Testdurchführung

3.1 Android Unit-Tests

Die Tests zeigen, dass zentrale Funktionen der Anwendung korrekt arbeiten. Abgedeckt werden insbesondere:

- Verarbeitung von Authentifizierungsdaten (JWT, Session-Handling)
- Speicherung von Einstellungen und Sitzungsdaten über SharedPreferences
- Konstruktion und Serialisierung von Data Classes und DTOs
- JSON-Verarbeitung (Serialisierung, Deserialisierung, Fehlerfälle)
- Fragen-Hierarchie und polymorphe Vererbung
- Zeit-, Alarm- und Locale-Verwaltung
- Utility- und Helper-Funktionen

Bei den vorliegenden Tests handelt es sich ausschließlich um lokale Unit-Tests, welche im Verzeichnis `src/test/unit` ausgeführt werden. Diese Tests sind bewusst plattformunabhängig gehalten und testen keine direkten Android-UI-Komponenten oder Framework-Klassen.

Android-spezifische Funktionalitäten wie Activities, Fragments, Navigation, Layouts oder reale Benachrichtigungen wurden nicht getestet, da diese typischerweise über Instrumentation-Tests (`src/androidTest`) abgesichert werden. Die Entscheidung, den Fokus auf Unit-Tests zu legen, ermöglicht schnelle, zuverlässige und reproduzierbare Tests der Kernlogik. Android-spezifische Abhängigkeiten (z.B. `Context`, `SharedPreferences`) werden mittels Mockito gemockt, um isolierte Tests ohne Android-Umgebung zu ermöglichen.

3.1.1 AlleApiTests.kt

Diese Testklasse enthält Unit Tests für verschiedene API-bezogene Komponenten unserer Android-Anwendung. Sie testet JWT-Verarbeitung, Session-Management und Data Classes für Authentifizierung und Protokoll-API. Getestete Komponenten sind *JWT Utils, Session, SessionStorage, Auth DTOsund Protocol DTOs*

Testfall	Beschreibung	Status
1. JWT ohne Username	Gibt null zurück bei JWT ohne username-claim	Bestanden
2. Session Management	Setzt und liest JWT in Session-Klasse	Bestanden
3. Session null setzen	Setzt Session-JWT korrekt auf null zurück	Bestanden
4. SessionStorage speichern	Speichert JWT in SharedPreferences (Mock)	Bestanden
5. SessionStorage laden	Lädt JWT aus SharedPreferences (Mock)	Bestanden
6. SessionStorage löschen	Löscht JWT aus SharedPreferences (Mock)	Bestanden
7. Login Request	Testet alle Properties der Login-Data-Class	Bestanden
8. Token Response	Testet TokenResponse Data Class Konstruktor	Bestanden
9. Protocol Answer DTO	Testet einfache ProtocolAnswerDto Data Class	Bestanden
10. Protocol Submission Request	Testet komplexe Request mit Antwortliste	Bestanden
11. Protocol Submission Response	Testet Response Data Class mit 3 Feldern	Bestanden
12. Data Class equals()	Vergleicht zwei identische Data Class Objekte	Bestanden
13. Data Class toString()	Überprüft String-Repräsentation der Data Class	Bestanden
14. Leere Antwort	Testet Edge Case mit leerem Ergebnis-String	Bestanden
15. Sonderzeichen Passwort	Testet Passwort mit Sonderzeichen	Bestanden
16. Cleanup	Bereinigt Session nach jedem Test	Bestanden

3.1.2 AlleDataServiceTests.kt

Diese Testklasse enthält Unit Tests für alle DataService-Komponenten der App. Sie testet Datenklassen, SharedPreferences-Operationen, JSON-Serialisierung und Fragen-Hierarchien. Getestete Komponenten sind *Answer.kt, DataLoader.kt, Question.kt, QuestionTypeAdapter.kt*

Testfall	Beschreibung	Status
1. Answer Konstruktor	Testet Answer-Data-Class mit id und result	Bestanden
2. QuestionSlider Properties	Testet alle Properties von QuestionSlider	Bestanden
3. QuestionTime Properties	Testet alle Properties von QuestionTime	Bestanden
4. QuestionDuration Properties	Testet alle Properties von QuestionDuration	Bestanden
5. QuestionMultipleChoice Properties	Testet Multiple-Choice-Fragen mit Auswahlmöglichkeiten	Bestanden
6. QuestionYesNo Properties	Testet Ja/Nein-Fragen mit skipIfNo Logik	Bestanden
7. QuestionYesNo Empty Skip	Testet Ja/Nein-Fragen ohne skipIfNo Liste	Bestanden
8. QuestionDate Properties	Testet alle Properties von QuestionDate	Bestanden
9. Question Vererbung	Prüft dass alle Fragetypen von Question erben	Bestanden
10. DataLoader Sprache speichern	Speichert Sprache in SharedPreferences	Bestanden
11. DataLoader Sprache laden	Lädt Sprache aus SharedPreferences	Bestanden
12. DataLoader Default Sprache	Lädt Standardsprache DE bei null	Bestanden
13. DataLoader Alarm Status speichern	Speichert Alarm-aktiviert Status	Bestanden
14. DataLoader Alarm Status laden	Lädt Alarm-aktiviert Status	Bestanden
15. DataLoader Alarm Zeit speichern	Speichert Alarmzeit als LocalTime	Bestanden
16. DataLoader Alarm Zeit laden	Lädt Alarmzeit aus SharedPreferences	Bestanden
17. DataLoader Default Alarmzeit	Lädt Standard-Alarmzeit (20:00)	Bestanden
18. DataLoader Farbschema speichern	Speichert Farbschema-Einstellung	Bestanden
19. DataLoader Farbschema laden	Lädt Farbschema aus SharedPreferences	Bestanden
20. DataLoader Default Farbschema	Lädt Standard-Farbschema system	Bestanden
21. Fragen-Datei nicht gefunden	Handhabt IOException bei fehlender JSON-Datei	Bestanden
22. Erster Start Default	Lädt Default-Wert für First-Time-Flag	Bestanden
23. FirstTime Completed setzen	Setzt First-Time-Flag auf false	Bestanden
24. Username speichern	Speichert Benutzername in SharedPreferences	Bestanden
25. Username laden	Lädt gespeicherten Benutzernamen	Bestanden
26. Gson Answer Serialisierung	Serialisiert Answer-Objekt zu JSON	Bestanden

27. Gson Answer Deserialisierung	Deserialisiert JSON zu Answer-Objekt	Bestanden
28. Gson Question Liste	Serialisiert Liste von Answer-Objekten	Bestanden
29. Gson QuestionSlider Roundtrip	Vollständiger Serialisierungs/Deserialisierungs-Test	Bestanden
30. QuestionTypeAdapter TypeMap	Testet TypeAdapter für Frage-Typen	Bestanden
31. Deserialisierung QuestionSlider	Deserialisiert QuestionSlider aus JSON	Bestanden
32. Deserialisierung MultipleChoice	Deserialisiert QuestionMultipleChoice aus JSON	Bestanden
33. Deserialisierung QuestionYesNo	Deserialisiert QuestionYesNo aus JSON	Bestanden
34. Deserialisierung QuestionTime	Deserialisiert QuestionTime aus JSON	Bestanden
35. Deserialisierung QuestionDuration	Deserialisiert QuestionDuration aus JSON	Bestanden
36. Deserialisierung QuestionDate	Deserialisiert QuestionDate aus JSON	Bestanden
37. Deserialisierung ungültiger Typ	Wirft Exception bei unbekanntem Typ	Bestanden
38. Deserialisierung ohne Typ	Wirft Exception bei fehlendem Typ-Feld	Bestanden
39. Deserialisierung leerer Typ	Wirft Exception bei leerem Typ-Feld	Bestanden
40. DataLoader Antworten speichern	Schreibt Antworten in Datei	Bestanden
41. Frage-Hierarchie	Testet Vererbungshierarchie aller Fragetypen	Bestanden
42. Answer Listen Operationen	Testet Listen-Operationen mit Answer-Objekten	Bestanden
43. Edge Cases - leere Strings	Testet mit leeren Strings in Antworten	Bestanden
44. Edge Cases - Sonderzeichen	Testet Umlaute und Sonderzeichen in Texten	Bestanden
45. Edge Cases - Maximalwerte	Testet mit Maximalwerten für IDs und Floats	Bestanden
46. Edge Cases - negative Werte	Testet mit negativen Werten für Slider	Bestanden
47. Liste contains Operation	Testet Listen-Operationen in QuestionYesNo	Bestanden
48. JSON Format mit Quotes	Testet JSON-Serialisierung mit Anführungszeichen	Bestanden

3.1.3 AlleHelperTests.kt

Diese Testklasse enthält Unit Tests für alle Helper-Klassen der App. Sie testet Notification-Logik, Alarm-Berechnungen, Locale-Verwaltung und verschiedene Hilfsfunktionen. Getestete Komponenten sind *NotificationHelper, AlarmReceiver (indirekt), updateLocale Funktion und Validierungs-Helfer*

Testfall	Beschreibung	Status
1. Alarm-Berechnung heute	Berechnet Alarmzeit für heute (Zeit in Zukunft)	Bestanden
2. Alarm-Berechnung morgen	Berechnet Alarmzeit für morgen (Zeit bereits vorbei)	Bestanden
3. Millisekunden Konvertierung	Konvertiert LocalDateTime zu Millisekunden	Bestanden
4. Alarm Edge Cases	Testet Mitternacht und gleiche Zeiten	Bestanden
5. AlarmReceiver Konstanten	Testet Notification-ID (hardcoded)	Bestanden
6. Notification Properties	Testet Channel-ID und Priority	Bestanden
7. updateLocale Deutsch	Testet deutsche Locale Erstellung	Bestanden
8. updateLocale Englisch	Testet englische Locale Erstellung	Bestanden
9. updateLocale Default	Testet Default Locale	Bestanden
10. Zeitvalidierung gültig	Testet gültige Uhrzeiten (00:00 - 23:59)	Bestanden
11. Zeitvalidierung ungültig	Testet ungültige Uhrzeiten (negative/zu große Werte)	Bestanden
12. Zeitformatierung	Formatiert Uhrzeiten als HH:mm und mit Ühr"	Bestanden
13. Datumsformatierung	Formatiert Datum als TT.MM.JJJJ	Bestanden
14. Intent Actions	Testet Android Intent-Action Konstanten	Bestanden
15. Package URI Format	Testet URI Format für Package-Namen	Bestanden
16. SharedPreferences Keys	Testet Konsistenz der SharedPreferences Keys	Bestanden
17. Mock Context	Testet Mock-Setup für Context und SharedPreferences	Bestanden
18. Android Version Checks	Testet Android SDK Version Konstanten	Bestanden
19. Alarm-Berechnung Helper	Testet Hilfsfunktion für Alarmzeit-Berechnung	Bestanden
20. Locale Funktionalität	Testet Locale-Klassenmethoden	Bestanden
21. String Resources Mock	Testet String-Resource-Namen	Bestanden
22. Edge Cases Zeiten	Testet Extremfälle für Uhrzeiten	Bestanden
23. Edge Cases Daten	Testet besondere Daten (Schaltjahre, 2038-Problem)	Bestanden
24. TimeZone Handling	Testet Zeitzonen-Verarbeitung	Bestanden
25. PendingIntent Flags	Testet Kombination von PendingIntent Flags	Bestanden

3.2 Android Integrationstests

Die durchgeführten Integrationstests folgen einer use-case-basierten Teststrategie, bei der typische Nutzungsszenarien der App mit der Backend-API simuliert werden. Jeder Test repräsentiert einen konkreten Anwendungsfall aus der Benutzerperspektive.

Technische Architektur:

- **MockWebServer:** Lokaler HTTP-Server von OkHttp, der API-Antworten simuliert
- **Retrofit:** HTTP-Client für API-Kommunikation
- **Gson:** JSON-Serialisierung/Deserialisierung
- **Kotlin Coroutines:** Asynchrone Ausführung der API-Aufrufe
- **Test-Location:** `src/test/integration/`

Test-Strategie:

- **Success-Fälle:** Korrekte Requests führen zu erwarteten Responses
- **Error-Fälle:** HTTP-Fehler (400, 401, 404) werden korrekt behandelt
- **Validierung:** Request-Methoden, Pfade, Headers und Body-Inhalte
- **Edge-Cases:** Ungültige JSON-Strukturen, fehlende Felder, leere Antworten

Setup-Prozess:

1. MockWebServer starten auf zufälligem Port
2. Retrofit-Instanz mit MockWebServer-URL konfigurieren
3. API-Interface über Retrofit instanziieren
4. Mock-Antworten mit `MockResponse` vorbereiten
5. Tests mit `runBlocking` für Coroutine-Support ausführen
6. MockWebServer nach jedem Test herunterfahren

3.2.1 AuthApiIntegrationTest.kt

Testet die Authentifizierungs-API-Endpunkte für Patient Login und First-Login.

Getestete Endpunkte:

- `POST /api/auth/patient/first-login` - Erstanmeldung mit Aktivierungscode
- `POST /api/auth/patient/login` - Reguläre Anmeldung mit Username/Password

Test-Szenarien:

- Erfolgreiche Authentifizierung mit korrektem Token
- Fehlerhafte Anfragen (ungültiger Code, falsche Credentials)
- Server-Fehler (400 Bad Request, 401 Unauthorized)
- Ungültige JSON-Responses

- Sequenzielle API-Aufrufe

Testfall	Beschreibung	Status
1. Patient First-Login Success	Erfolgreiche Erstanmeldung mit Token-Rückgabe	Bestanden
2. Patient Login Success	Erfolgreiche reguläre Anmeldung	Bestanden
3. First-Login Invalid Code	HTTP 400 bei ungültigem Aktivierungscode	Bestanden
4. Login Wrong Credentials	HTTP 401 bei falschen Anmeldedaten	Bestanden
5. Invalid JSON Response	Exception bei ungültiger JSON-Struktur	Bestanden
6. Multiple Sequential Requests	Korrekte Verarbeitung sequentieller Aufrufe	Bestanden

3.2.2 ProtocolApiIntegrationTest.kt

Testet das Absenden von ausgefüllten Schlafprotokollen an die Backend-API.

Getesteter Endpunkt:

- `POST /api/protocols/submissions` - Übermittlung ausgefüllter Protokolle

Test-Szenarien:

- Erfolgreiche Protokoll-Übermittlung mit Submission-ID
- Ungültige Protokolldaten (HTTP 400)
- Fehlende/ungültige Authentifizierung (HTTP 401)
- Validierung von Request-Body und JSON-Struktur

Testfall	Beschreibung	Status
1. Submit Protocol Success	Erfolgreiche Übermittlung mit Submission-ID	Bestanden
2. Submit Invalid Data	HTTP 400 bei ungültigen Protokolldaten	Bestanden
3. Submit Unauthorized	HTTP 401 bei fehlender Authentifizierung	Bestanden

3.2.3 ReportApiIntegrationTest.kt

Testet den Download von PDF-Berichten mit verschiedenen Parametern.

Getesteter Endpunkt:

- `GET /api/reports/me/pdf` - Download persönlicher PDF-Berichte

Test-Szenarien:

- Erfolgreicher PDF-Download mit korrekten Headers
- Verschiedene Query-Parameter (Date-Range, days, allSubmissions)

- Fehlende Authentifizierung (HTTP 401)
- Keine Daten verfügbar (HTTP 404)
- Validierung von Content-Type und Content-Disposition Headers

Testfall	Beschreibung	Status
1. PDF Download Default	Erfolgreicher Download mit Standard-Parametern	Bestanden
2. PDF with Date Range	Download mit spezifischem Zeitraum	Bestanden
3. PDF Unauthorized	HTTP 401 bei fehlender Authentifizierung	Bestanden
4. PDF No Data	HTTP 404 wenn keine Daten vorhanden	Bestanden

3.3 Backend-Modultests des Schlaftagebuch-Systems (Spring Boot)

3.3.1 Getestete Module und Use Cases

Die Tests befinden sich unter `test/java/de.schlaftagebuch.schlaftagebuch_backend/[Modul]` und umfassen die vier Kernmodule des Systems:

Modul	Funktionalität
Auth	Arzt-/Patient-Login, Registrierung, JWT-Token, Passwort-Verschlüsselung
ArztManagement	Code-Generierung für Patienten, Arzt-Patienten-Zuordnung, Code-Verwaltung
Protocol	Erfassung und Verwaltung von Schlaftagebuch-Einträgen (CRUD)
Report	PDF-Generierung, Datenanalyse, Visualisierung, Export

Tabelle 1: Getestete Module entsprechend der Use Cases

Diese Module bilden die in den Use Cases definierten Hauptfunktionalitäten ab und enthalten die kritischen Sicherheits- und Geschäftslogik-Komponenten.

3.3.2 Teststrategie

Die Testimplementierung basiert auf einer **rein Unit-Test-basierten Strategie** mit Mockito. Services und Controller werden isoliert getestet, wobei alle Abhängigkeiten (Repositories, externe Services) gemockt werden. Diese fokussierte Herangehensweise ermöglicht schnelle Testausführungen und konzentriert sich auf die Validierung der Business-Logik unter kontrollierten Bedingungen.

3.3.3 Testszenarien

Die gewählten Unit-Test-Szenarien decken die Kernfunktionalitäten umfassend ab:

- **Positivfälle:** Testung aller erwarteten Erfolgsszenarien mit gültigen Eingabedaten
- **Negativfälle:** Systematische Testung von Grenzwerten und ungültigen Eingaben (24% der Tests)
- **Edge Cases:** Behandlung von Sonderfällen wie Null-Werten, leeren Listen, extremen Datenmengen
- **Fehlerbehandlung:** Verifikation der korrekten Exception-Propagation von Service- zu Controller-Schicht
- **Verhaltensverifikation:** Überprüfung der korrekten Interaktion mit gemockten Abhängigkeiten

3.3.4 Technische Infrastruktur

- Spring Boot 3.4.0, Java 17
- **JUnit 5 + Mockito 5.x** für reine Unit-Tests
- AssertJ für flüssige Assertions
- Gradle mit JaCoCo Plugin für Coverage-Analyse

3.3.5 Ausführung

```
# Alle Tests
./gradlew clean test

# Mit Coverage-Report
./gradlew test jacocoTestReport
```

Report: `build/reports/jacoco/test/html/index.html`

3.3.6 Testergebnisse

Getesteter Funktionsbereich	**Status**
Arzt-Login (Erfolg, falsche Daten, falsche Rolle)	✓
Patienten-Login (Erfolg, falsche Daten, falsche Rolle)	✓
Einmal-Code-Erzeugung (neu / bestehender Patient)	✓
Patienten-Erstlogin mit Einmal-Code	✓
Validierung von Codes (ungültig, abgelaufen, benutzt, widerrufen)	✓
Validierung von Eingabedaten (Geburtsdatum, Consent)	✓
JWT-Token-Erstellung und Rollenprüfung	✓
AuthController-Endpunkte (Service-Delegation, Fehlerweitergabe)	✓

Tabelle 2: Testergebnisse des Auth-Moduls

Anzahl Unit-Tests (Service)	21
Anzahl Unit-Tests (Controller)	5
Bestandene Tests	26(100%)
Fehlgeschlagene Tests	0
Code Coverage (Service-Schicht)	75%

Tabelle 3: Zusammenfassung der Testergebnisse für das Auth-Modul

Getesteter Funktionsbereich	**Status**
Verwaltung von Patienten- und Arzt-Zuordnungen	✓
Behandlung fehlender oder ungültiger Patientendaten	✓
Generierung, Statusverwaltung und Deaktivierung von Einmal-Codes	✓
Berechnung und Rückgabe von Dashboard-Statistiken	✓
Delegation der REST-Endpunkte an die jeweilige Service-Schicht	✓

Tabelle 4: Testergebnisse des ArztManagement-Moduls

Metrik	**Wert**
Anzahl Unit-Tests (ArztDashboardService)	10
Anzahl Unit-Tests (ArztManagementService)	20
Anzahl Unit-Tests (ArztManagementController)	8
Bestandene Tests	38 (100%)
Fehlgeschlagene Tests	0
Code Coverage (Service-Schicht)	87%

Tabelle 5: Zusammenfassung der Testergebnisse für das ArztManagement-Modul

Getesteter Funktionsbereich	Status
Verarbeitung von Protokoll-Einreichungen verschiedener Benutzertypen	✓
Validierung gültiger Anfragen	✓
JSON-Serialisierung mit Sonderzeichen und Escaping	✓
Unterstützung verschiedener Protokoll-Vorlagen und Sprachen	✓
Automatische Zeitstempel-Generierung bei fehlenden Daten	✓
Behandlung von Edge Cases und großen Datenmengen	✓
Korrektes Mapping zwischen Entity und Response-DTO	✓
Exception-Propagation von Service zu Controller	✓

Tabelle 6: Testergebnisse des Protocol-Moduls

Im Rahmen der Testentwicklung wurden zusätzlich Sonderfälle mit `null`- oder leeren Eingaben berücksichtigt, um die Vollständigkeit der Testbetrachtung sicherzustellen.

- Übergabe eines `null`-Request-Objekts
- Übergabe einer `null`-Antwortliste
- Übergabe einer leeren Antwortliste

Diese Szenarien treten im produktiven Betrieb nicht auf, da die Request-Objekte in der Controller-Schicht durch Spring Boot erzeugt, validiert und mit Default-Werten initialisiert werden.

Hinweis: Die genannten Sonderfälle dienen der theoretischen Robustheitsbetrachtung und haben keine Auswirkung auf den realen Betrieb des Systems.

Metrik	Wert
Anzahl Unit-Tests (Service)	21
Anzahl Unit-Tests (Controller)	8
Gesamtzahl Unit-Tests	29
Bestandene Tests	26 (90%)
Fehlgeschlagene Tests	3
Code Coverage (Service-Schicht)	94%
Code Coverage (Controller-Schicht)	89%

Tabelle 7: Zusammenfassung der Testergebnisse für das Protocol-Modul

Getesteter Funktionsbereich	Status
Report-Generierung für Patienten (eigene Berichte)	✓
Report-Generierung für Ärzte (Patientenberichte)	✓
PDF-Export von Schlaftagebuch-Berichten	✓
Zeitraum-basierte Report-Generierung (Date Range)	✓
Tages-basierte Report-Generierung (Last N Days)	✓
Automatische Patientenerkennung aus Authentifizierung	✓
Behandlung verschiedener Eingabeformate (JSON)	✓
Validierung von Eingabeparametern	✓
Berechnung von Schlafstatistiken	✓
Integration mit FHIR für medizinische Berichte	✓

Tabelle 8: Testergebnisse des Report-Moduls

Metrik	Wert
Anzahl Unit-Tests (Service)	23
Anzahl Unit-Tests (Controller)	9
Gesamtzahl Unit-Tests	32
Bestandene Tests	32 (100%)
Fehlgeschlagene Tests	0
Code Coverage (Service-Schicht)	94%
Code Coverage (Controller-Schicht)	89%

Tabelle 9: Zusammenfassung der Testergebnisse für das Report-Modul

3.4 Test mit Postman

Im Folgenden wird die Testdurchführung der relevanten API-Endpunkte mit Postman dokumentiert. Die Tests erfolgen lokal gegen `http://localhost:8080`. Für alle Requests mit JWT ist sicherzustellen, dass das Token vollständig kopiert und als Bearer Token gesetzt wird.

3.4.1 Als Arzt einloggen

Methode: `POST`
URL: `http://localhost:8080/api/auth/arzt/login`

Body (JSON):

```
{
  "username": "arzt1",
  "password": "arztpass"
}
```

Erwartete Antwort: Es wird ein JWT zurückgegeben, z. B.:

```
{ "token": "eyJhbGciOi..." }
```

Der Token muss vollständig kopiert werden.

3.4.2 Zugangscode generieren

Methode: `POST`
URL: `http://localhost:8080/api/auth/patients/generate-code`

Auth in Postman: `Auth` → `Bearer Token` und das zuvor erhaltene Arzt-JWT einfügen.
Body: kein Body

Erwartete Antwort: `GenerateCodeResponse` mit `code` (und ggf. `patientId`).

3.4.3 Erster Login des Patienten

Methode: `POST`
URL: `http://localhost:8080/api/auth/patient/first-login`

Body (JSON, Code ersetzen):

```
{
  "code": "MQ4842",
  "consentAccepted": true,
  "givenName": "Test",
  "familyName": "Patient",
  "password": "pass123",
  "birthDate": "2000-01-01"
}
```

Ergebnis: Der Patient wird initial registriert/aktiviert (abhängig von Backend-Implementierung) und kann anschließend authentifiziert werden.

3.4.4 Test-Login Patient (über Seed, DEV)

Hinweis: Funktioniert aktuell *ohne* Authentifizierung und dient ausschließlich Testzwecken in der Entwicklungsumgebung.
Methode: `POST`
URL: `http://localhost:8080/api/auth/patient/login`

Body (JSON):

```
{
  "username": "patient_test",
  "password": "patientpass"
```

```
}
```

Erwartete Antwort: Enthält das JWT des Patienten (Patient-Token), welches für die folgenden geschützten Endpunkte verwendet wird.

3.4.5 Protokoll absenden

Methode: `POST`
URL: `http://localhost:8080/api/protocols/submissions`

Authorization Header: `Authorization: Bearer <PATIENT_TOKEN>`

Body (JSON):

```
{
  "templateKey": "schlaftagebuch_v1",
  "locale": "de",
  "filledAt": "2025-12-30T10:15:00Z",
  "answers": [
    { "id": 0, "result": "7" },
    { "id": 1, "result": "22:45" }
  ]
}
```

Erwartetes Ergebnis: Wenn eine `submissionId` zurückkommt, wurde das Protokoll gespeichert und dem Patienten zugeordnet.

3.4.6 Bericht erstellen (PDF) – Test

Methode: `GET`
URL: `http://localhost:8080/api/reports/me/pdf?days=14`

Authorization Header: `Authorization: Bearer <PATIENT_TOKEN>`

Ergebnis: Es wird ein PDF-Bericht für den Patienten generiert (Zeitraum: letzte `days=14` Tage), sofern entsprechende Daten vorhanden sind.

3.5 Query-Parameter

Warum benötigen wir einen Query-Parameter?
Im Produktivbetrieb ist es sinnvoll, im PDF-Bericht pro Kalendertag ausschließlich den jeweils neuesten Schlafprotokoll-Eintrag zu berücksichtigen. Dadurch bleibt der Bericht übersichtlich, gut vergleichbar und medizinisch sinnvoll interpretierbar.

Für Test- und Entwicklungszwecke ist dieses Verhalten jedoch unpraktisch, da man ansonsten viele Kalendertage abwarten müsste, um ausreichend Daten für die Berichtserstellung und Visualisierung zu erzeugen.

Aus diesem Grund wird ein zusätzlicher **Query-Parameter** eingeführt, mit dem das Verhalten der Berichtserstellung gezielt zwischen Produktiv- und Testmodus umgeschaltet werden kann.

- `allSubmissions=false` (Standardbetrieb):
  Pro Kalendertag wird nur die neueste Submission im Bericht berücksichtigt. Dieses Verhalten ist für den regulären Einsatz vorgesehen.
- `allSubmissions=true` (Testmodus):
  Alle Submissions im gewählten Zeitraum werden berücksichtigt, auch wenn mehrere Einträge am selben Tag existieren.

Wie funktioniert das technisch?
Ein Query-Parameter ist ein optionaler Parameter, der bei einem HTTP-Request an den Endpoint angehängt wird. Er beginnt mit einem Fragezeichen (`?`); mehrere Parameter werden mit einem kaufmännischen Und-Zeichen (`&`) getrennt.

Im Backend wird der Parameter über das Spring-Framework mit `@RequestParam` entgegengenommen und steuert dort die interne Logik der Datenaggregation.

Relevanter Endpoint

Der PDF-Bericht wird über folgenden Endpoint erzeugt:

```
GET /api/reports/me/pdf
```

Beispiele für die Verwendung

- **Standardbetrieb (Produktivmodus, 14 Tage):**

```
/api/reports/me/pdf?days=14
```

- **Testmodus (alle Submissions im Zeitraum):**

```
/api/reports/me/pdf?days=14&allSubmissions=true
```

- **Schnelltest (ein Tag, mehrere Submissions):**

```
/api/reports/me/pdf?days=1&allSubmissions=true
```

Definition im Backend
Der Parameter ist im Controller wie folgt definiert:

```
@RequestParam(defaultValue = false") boolean allSubmissions
```

Dadurch ist sichergestellt, dass ohne explizite Angabe automatisch das Standardverhalten (Produktivmodus) verwendet wird.

Konkretes Testvorgehen

1. **Erzeugen mehrerer Submissions:**
   Das Schlafprotokoll innerhalb der mobilen Anwendung mehrfach am selben Tag ausfüllen und absenden.
2. **Abrufen des PDF-Berichts im Testmodus:**
   Der Bericht wird innerhalb der App mit aktivierter Testkonfiguration (`allSubmissions=true`) generiert.
3. **Erwartetes Ergebnis:**
   Im generierten PDF erscheinen mehrere Einträge für denselben Kalendertag. Die Diagramme enthalten entsprechend mehrere Datenpunkte mit eindeutig unterscheidbaren Beschriftungen.
4. **Rückkehr zum Standardbetrieb:**
   Wird der Parameter `allSubmissions` nicht gesetzt oder auf `false` belassen, enthält der Bericht wieder ausschließlich den jeweils neuesten Eintrag pro Tag.

4 Installations- und Bedienungsanleitung

4.1 Zielgruppe und Zweck

Diese Anleitung richtet sich an technische Nutzer:innen (Tutor:innen, Reviewer, Entwickler:innen), die das System lokal in Betrieb nehmen, testen und demonstrieren möchten. Sie beschreibt die Inbetriebnahme und Bedienung der folgenden Komponenten:

- Backend (Spring Boot, REST-API) inkl. Datenbank via Docker
- Android-App (Kotlin / Jetpack Compose)
- Web-Anwendung (Web-GUI, Anbindung an das Backend über REST)
- Optional: Smartwatch / Wear OS (Datenquelle / Synchronisation)

4.2 Rollen und Berechtigungen (Kurzüberblick)

- **Patient (PATIENT):** Erstzugang über Einladungs- bzw. Zugangscode, Erfassung eigener Daten, kein Zugriff auf fremde Daten.
- **Arzt/Ärztin (ARZT):** Anmeldung mit bereitgestelltem Konto, Erstellung von Zugangscodes, Zugriff nur auf zugewiesene Patient:innen, Auswertung und Report-Erstellung.

4.3 Voraussetzungen (Software)

- **Docker Desktop** inkl. Docker Compose (Datenbank)
- **Java JDK 17**
- **Android Studio** (für die Android-App)
- **Visual Studio Code** (für die Web-Komponente)

4.4 Backend: Infrastruktur und Server starten

Start der Datenbank und abhängiger Services in Docker-Containern.

1. Docker Desktop öffnen und sicherstellen, dass Docker läuft (Status: *running*).
2. Terminal öffnen und ins Projektverzeichnis wechseln:

   `cd <pfad-zum-repo>/src/back`
3. Container starten:

   `docker compose up -d`

Erwartetes Ergebnis:

- Container laufen stabil ohne Crash (sichtbar in Docker Desktop).
- Datenbank ist erreichbar.

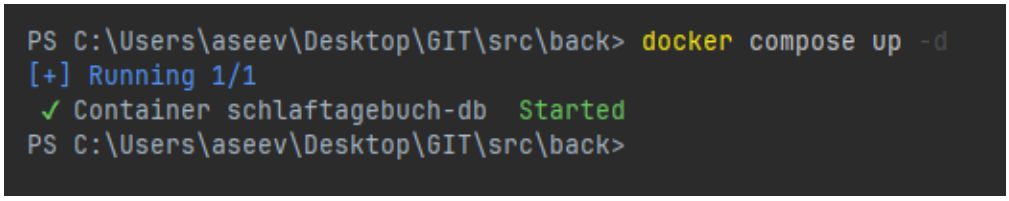


Abbildung 20: Erfolgreicher Start der Datenbank über Docker Compose

Datenbank prüfen (optional) **Variante A: Docker Desktop**

- Running Containers → Datenbank-Container → Logs prüfen (keine Crash-Loops).

Variante B: DB-Client (IntelliJ Database / pgAdmin)

- Host: `127.0.0.1`
- Port: `5433`
- Credentials: gemäß `docker-compose.yml` / `.env`
- Erwartung: Tabellen/Schemata sind sichtbar.

REST-API lokal starten

1. Terminal im Backend-Verzeichnis öffnen.
2. Server starten:

   `.\gradlew bootRun`

   (nachdem die Datenbank mit `docker compose up -d` gestartet wurde)
3. Alternativ: Start über IDE (Run-Konfiguration).

Erwartetes Ergebnis (Erfolgsindikatoren):

- Im Terminal erscheinen Hibernate-Logausgaben (z. B. SQL-Selects auf die Tabelle `user_account`) sowie der Status `EXECUTING :bootRun`.
- Es treten keine roten Stacktraces oder Abbruchmeldungen auf.
- Der Prozess bleibt aktiv und wartet auf eingehende Requests.

Hinweis: Die angezeigten Hibernate-Abfragen sowie der laufende `bootRun`-Prozess zeigen, dass das Backend erfolgreich gestartet ist und betriebsbereit ist. Der Gradle-Fortschritt verbleibt typischerweise bei ca. 80 %.

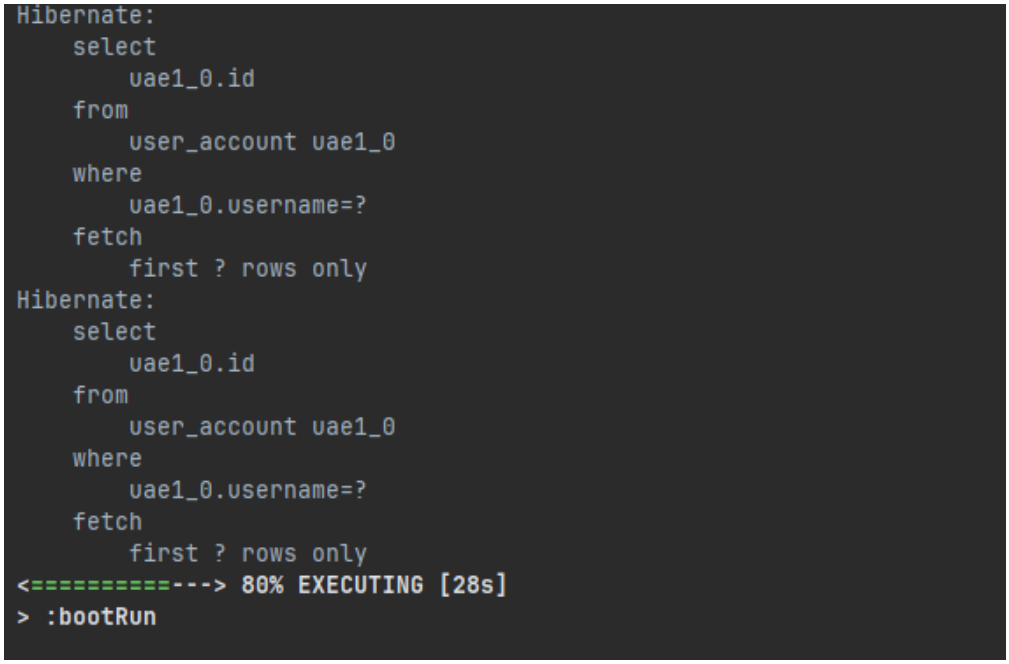


Abbildung 21: Erfolgreicher Start des Backends mit laufendem `bootRun`-Prozess

Überprüfung der Funktionsfähigkeit:

- Eine Überprüfung über den Webbrowser ist nicht möglich, da das Backend ausschließlich REST-Endpunkte bereitstellt.
- Die Funktionsfähigkeit kann entweder über ein API-Testtool (z. B. Postman) oder über die angebundenen Frontends (Android-App oder Web-GUI) überprüft werden.

4.5 Web-Anwendung

1. Visual Studio Code öffnen.
2. Extension *Live Server* installieren.
3. Web-Ordner öffnen (`src/HTML`).
4. `login.html` öffnen.
5. Rechtsklick → *Open with Live Server*.

Erwartetes Ergebnis:

- Browser öffnet eine lokale URL (z. B. `http://127.0.0.1:5500/`).
- Seite wird korrekt gerendert, keine 404 im Browser.

Authentifizierungspfad prüfen.

1. **Login-Seite:** Benutzername/Passwort eingeben
   (Benutzername: arzt1, Passwort: arztpass)
   → erfolgreicher Übergang zur Hauptübersicht.
2. **Navigation:** Über Sidebar sind alle anderen Seiten zu erreichen → Äbmeldenüm zur Login-Seite zu kommen.
3. **Hauptübersicht:** Anzahl der Patient:innen und Berichte, die zur Verfügung stehen sowie letzte Aktivitäten einsehen.
4. **Patient:innen-Seite:** Patientendaten beinhalten Geburtsdatum, Zugangscode, Termin und Bericht.
5. **Zugangscodes-Seite:** Erstellung von Zugangscode (Voraussetzung: Patient gewählt, Gültigkeitsdatum gewählt, Termin gewählt).
6. **Berichte-Seite:** Nach 14 Tagen können hier Berichte von Patient:innen im PDF-Format heruntergeladen werden.

4.6 Android: App starten und Grundfunktion prüfen

Variante A: Emulator

1. Android Studio → *Device Manager*.
2. Emulator erstellen (z. B. Pixel-Gerät, Android-Version gemäß Projektvorgaben).
3. Emulator starten.

Variante B: Physisches Gerät

1. Auf dem Android-Gerät die Entwickleroptionen aktivieren (in den Einstellungen 7x auf „Build-Nummer“ tippen z.B. unter Einstellungen/ Telefoninfo/ Softwareinformation/ Buildnummer).
2. In den Entwickleroptionen **USB-Debugging** aktivieren.
3. Gerät via USB mit dem PC verbinden und den RSA-Fingerprint (angezeigt Meldung) bestätigen.
4. In Android Studio im Dropdown-Menü das physische Gerät auswählen.

In Android-Studio:

1. Das App-Modul auswählen.
2. Zielgerät wählen (Emulator oder physisches Gerät).
3. *Run* starten.

Erwartetes Ergebnis:

- App wird installiert und startet ohne Crash.
- Sprache-/Login-Screen/Menü erscheint.

Nachweis: Screenshot vom Start-/Login-Screen (Abb. __) und ggf. Logcat-Ausschnitt ohne Fehler.

Authentifizierungspfad prüfen.

- **Patient First Login:** Die Sprache auswählen → Konto erstellen → Einladungscode eingeben und PbD eingeben (und ggf. Zusatzdaten) → Account-Aktivierung → Hauptansicht.
- **Einstellungen verwalten:** Menü →Knopf mit Einstellungen → → Änderung vornehmen (z. B. Erinnerungszeit) → Zurück zur Hauptansicht (Änderung ist gespeichert).
- **Smartwatch koppeln:** Bluetooth Einstellungen → Falls Smartwatch nicht angezeigt dannn Geräte suchen → Mit Smartwatch verbinden → App auf der Smartwatch öffnen → Gibt Feedback bei Schlaf Starten.

- **Protokoll ausfüllen:** Menü → Knopf mit Fragebogen starten → Fragen mit weiter durchklicken
- **Bericht ansehen:** Menü → Button mit Bericht herunterladen (PDF) → Es öffnet sich eine PDF (oder Fehler weil kein Server)

Erwartetes Ergebnis:

- Erfolgreiche Anmeldung führt zur jeweiligen Hauptansicht.
- Bei ungültigen Eingaben erscheint eine verständliche Fehlermeldung.

4.7 Smartwatch / Wear OS (optional)

Variante A: Wear OS Emulator

1. Android Studio → *Device Manager*.
2. Wear OS Emulator erstellen und starten.

Variante B: Physische Smartwatch

1. Auf der Smartwatch: **Entwickleroptionen** aktivieren (Einstellungen → System → Info → 7x auf „Build-Nummer“ tippen).
2. In den Entwickleroptionen **ADB-Debugging** und **Debugging über WLAN** aktivieren.
3. In Android Studio: *Device Manager* öffnen → *Pair using Wi-Fi*.
4. Den Anweisungen auf Uhr und PC folgen (QR-Code scannen oder Code eingeben).

Erwartetes Ergebnis:

- Wearable wird erkannt.
- Synchronisation kann angestoßen werden (falls implementiert).

Nachweis: Screenshot der Wear-Ansicht oder Logcat-Eintrag der Synchronisation (Abb. __).

5 Quellenangabe (Informationsbeschaffung / Dokumentation)

Die folgende Liste enthält die im Rahmen des Projekts verwendeten offiziellen Dokumentationen und Ressourcen zur Informationsbeschaffung.

- **Android / Mobile App**
  - Jetpack Compose – Offizielle Dokumentation:
    `https://developer.android.com/compose`
  - Material Design 3 – Komponenten:
    `https://m3.material.io/components`
- **Backend / Server**
  - Spring Boot – Offizielle Dokumentation:
    `https://spring.io/projects/spring-boot`
- **Datenbank**
  - PostgreSQL – Offizielle Dokumentation:
    `https://www.postgresql.org/docs/`
- **Containerisierung**
  - Docker – Offizielle Dokumentation:
    `https://docs.docker.com/`
- **API-Tests**
  - Postman – Offizielle Dokumentation:
    `https://learning.postman.com/`
- **Web-Technologien**
  - HTML – Dokumentation:
    `https://www.w3schools.com/html/`
  - JavaScript – Dokumentation:
    `https://www.w3schools.com/js/`
  - CSS – Grundlagen (SelfHTML Wiki):
    `https://wiki.selfhtml.org/wiki/CSS`

5.1 Verwendete Tools und Programme

Im Rahmen des Projekts wurden folgende Tools und Programme eingesetzt:

- **Entwicklung und Programmierung**
  - Android Studio
  - IntelliJ IDEA
  - Visual Studio Code
- **Versionsverwaltung**
  - Git
    `https://git-scm.com`
  - GitLab (Repository und CI/CD)
- **Backend und Infrastruktur**
  - Spring Boot
  - Docker
  - PostgreSQL
  - Gradle
- **API-Tests**
  - Postman
- **Diagramme und Dokumentation**
  - draw.io (Diagrammerstellung)
    `https://app.diagrams.net`
  - Affinity Publisher (Layout und PDF-Erstellung)
    `https://www.affinity.serif.com/de/publisher/`
  - LaTeX
    `https://www.latex-project.org/help/documentation/`
- **Medien und Präsentation**
  - Clipchamp (Video-Editing)
    `https://clipchamp.com/de/video-editor/`